

Week 1: Introduction to Computer Network

การส่งจดหมายในประเทศ vs การส่งจดหมายไปต่างประเทศ ต่างกันยังไง ใช้ protocol อะไร และมี entity ที่เกี่ยวข้องอะไรบ้าง?

องค์ประกอบการสื่อสาร

- ฝ่ายส่ง = อุปกรณ์ที่ใช้ส่งข้อมูล (Computer) & อุปกรณ์ที่ใช้ในการสื่อสาร เพื่อเชื่อมกับช่องทางการสื่อสาร
Ex. Modem
- ช่องทางการสื่อสาร หรือตัวกลางสำหรับการส่งสัญญาณ Ex. สายโทรศัพท์ หรือ สัญญาณ วิทยุ
- ฝ่ายรับ = อุปกรณ์ที่ใช้รับข้อมูล (Computer) & อุปกรณ์ที่ใช้ในการสื่อสาร เพื่อเชื่อมกับช่องทางการสื่อสาร Ex. Modem

อุปกรณ์ในการสื่อสาร

ในเครื่อง : การ์ดแลน / การ์ด WiFi / การ์ด Bluetooth

นอกเครื่อง : โมเด็ม / Router / AP

ตัวกลาง

แบบใช้สาย :

- สายเกลียวพันคู่/สายคู่บิดเกลียว >
 - 1.สาย UTP (Unshielded Twisted Pair) ex. สายโทรศัพท์, สายแลน (ทั่วไปใช้แบบนี้)
 - 2.สาย STP (Shielded Twisted Pair) ex. สายโทรศัพท์, สายแลน (ที่ต้องการลดสัญญาณรบกวน)
- สายโคแอกเชียล
- สายใยแก้วนำแสง

แบบไร้สาย : สัญญาณรบกวนมากกว่า

- คลื่นวิทยุ = ส่งเป็นวงกลม ex. Bluetooth, WiFi, Wimax
- คลื่นโทรศัพท์มือถือ = คลื่นวิทยุที่ความถี่เฉพาะ ex. 2G (GPRS, EDGE), 3G
- คลื่นไมโครเวฟ = ส่งตรงๆ ex. สัญญาณดาวเทียม, สัญญาณ GPS
- อินฟราเรด = ส่งตรงๆ ex. รีโมททีวี

Bandwidth = จำนวนข้อมูลที่สามารถเดินทางได้บนช่องทางการสื่อสาร ก็คือส่งได้เร็วแค่ไหน
มีหน่วยเป็น bps => bit per sec

Q1-2

1. ถ้าทั้งสองเครื่องมีการ์ดแลน แล้วต่อสายทั้งสองเครื่องด้วย Ethernet cable ทั้งสองเครื่องนี้สามารถส่งข้อมูลหากันได้เลยหรือไม่ อย่างไร
= ไม่ได้ ต้องมีการ configure ก่อน เช่นการ set ip ของเครื่อง
2. ถ้าทั้งสองเครื่องต่อกันและคุยกันได้ เป็น ระบบเครือข่าย (Computer Network) หรือไม่อย่างไร
= เป็น เนื่องจากระบบเครือข่ายคือการเชื่อมต่อระหว่างอุปกรณ์สองเครื่องหรือมากกว่า เพื่อให้สามารถสื่อสารและแลกเปลี่ยนข้อมูลกันได้ ไม่ว่าจะเป็นคอมพิวเตอร์ โทรศัพท์มือถือ หรืออุปกรณ์อื่นๆ ที่เชื่อมต่อกันผ่านสายหรือไร้สาย

ระบบเครือข่ายคอมพิวเตอร์ (Computer Network) = ระบบที่มีการนำเอาคอมพิวเตอร์หลายๆ เครื่องขึ้นไปมาเชื่อมต่อเข้าไว้ด้วยกัน เพื่อแลกเปลี่ยนข้อมูลข่าวสาร รวมถึงการใช้ทรัพยากรบางอย่างของระบบร่วมกันได้

Q3-4

3. ทั้งสองเครื่องนี้สามารถต่อ internet ไปหาอีกเครื่องที่อยู่กันคนละที่ได้หรือไม่
= ไม่ได้ เพราะอยู่คนละ network
4. ถ้าไม่ ต้องมีการติดตั้งอุปกรณ์ใดเพิ่มในระบบเครือข่าย
= ติดตั้ง Router
5. จัตุการต่อ internet วิธีต่างๆ กับรูปต่อไปนี้



1. ต่อโดยใช้ **Sim Card**

2. ต่อที่บ้าน ที่ทำการต่อ **Internet** ความเร็วสูง กับ **ISP** เช่น **3BB /AIS**

3. ต่อที่คณะ

What is router?

- อุปกรณ์ที่ทำหน้าที่ในการเชื่อมต่อระบบเครือข่ายคนละระบบเข้าด้วยกัน
- อุปกรณ์ที่เชื่อม computer หลายๆ เครื่องเข้าด้วยกัน และทำการส่งผ่านข้อมูลระหว่างอุปกรณ์ไปยังอุปกรณ์ปลายทางที่ต้องการ
- พบได้ทุกขนาดของ network ex. บ้าน, บริษัท, มหาวิทยาลัย

LANs (Local Area Network)

- เชื่อมต่อคอมพิวเตอร์ในระยะจำกัด ex. บ้าน, อาคารเดียวกัน (ปก. 1Gbps)
- เชื่อมด้วยสายเคเบิล (สาย LAN) RJ45 or CAT... เรียกว่า Ethernet > 100Mbps, 1Gbps, 10Gbps
- เชื่อมต่อแบบไร้สาย WLAN หรือ WiFi : wireless access points at 11, 54, 450 Mbps

MANs (Metropolitan Area Network) => ระดับเมือง

WANs (Wide Area Network)

- ครอบคลุมพื้นที่ขนาดใหญ่ ex. ประเทศ, โลก เชื่อมด้วยระบบสื่อสารหลายแบบ ทั้งสายโทรศัพท์, สายเคเบิล, คลื่นวิทยุ
- WAN เป็นได้ทั้งระบบเครือข่ายใหญ่ๆ ระบบเดี่ยวหรือประกอบไปด้วย หลายๆ LANs ด้วยกัน

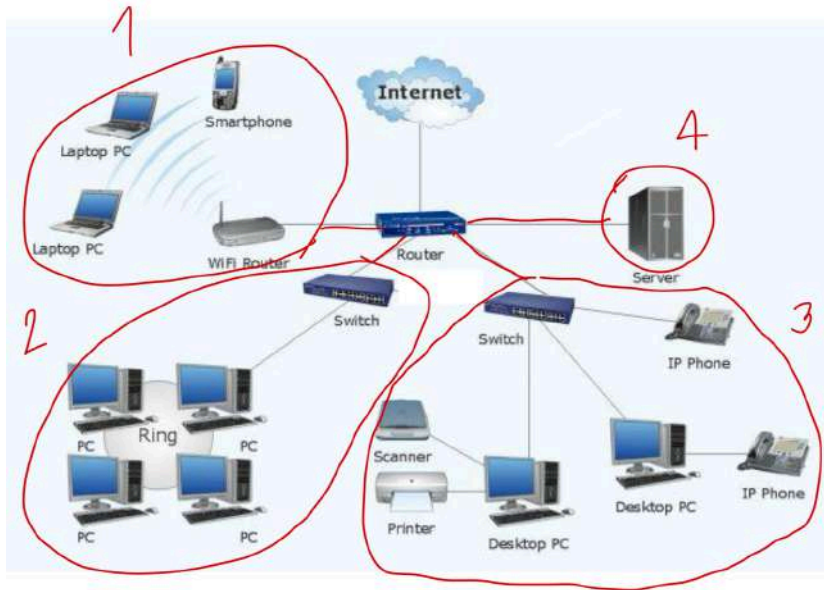
Q6 : WAN ที่ใหญ่ที่สุดในโลก = Internet

Network ที่ไม่ต่อ internet = Intranet

Extranet : เป็นประเภทหนึ่งที่ยอมให้เข้าถึงข้อมูลบางส่วน ex. Bank ก้

น Broker

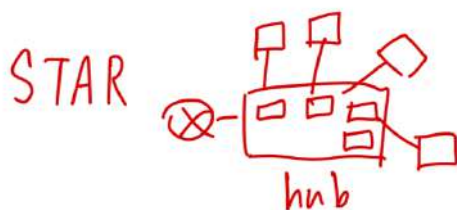
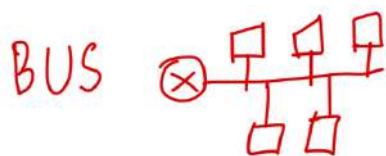
Q7: ในรูปมีทั้งหมดกี่วง LAN = 4 วง



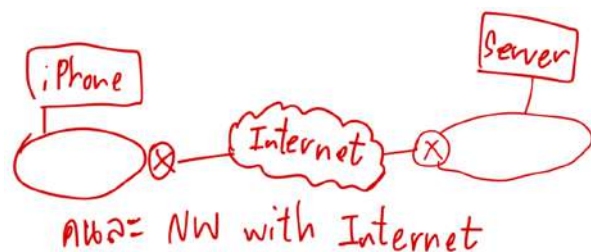
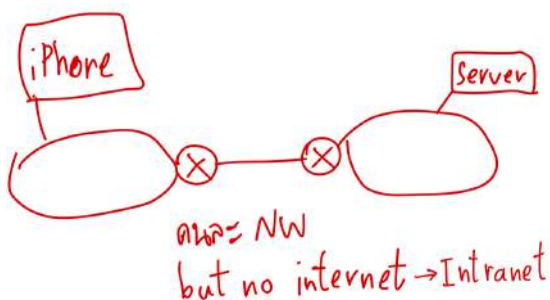
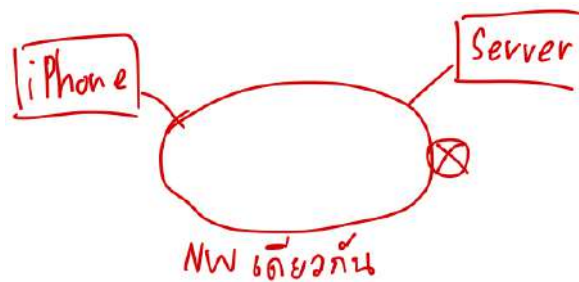
How to วาด network configuration แบบสากล

- ให้วาดวง LAN ด้วยวงรี
- Router ให้ใช้สัญลักษณ์ router ของ cisco
- Internet ให้ใช้สัญลักษณ์รูปเมฆ และเขียน Internet อยู่ตรงกลาง

Topology = การต่อของเครื่องใน network



BUS : ถ้าเส้นหลักเสีย ดับทั้งวง, STAR เสียเป็นส่วนๆ ได้
ปกติทั่วไปที่เจอเป็นแบบนี้

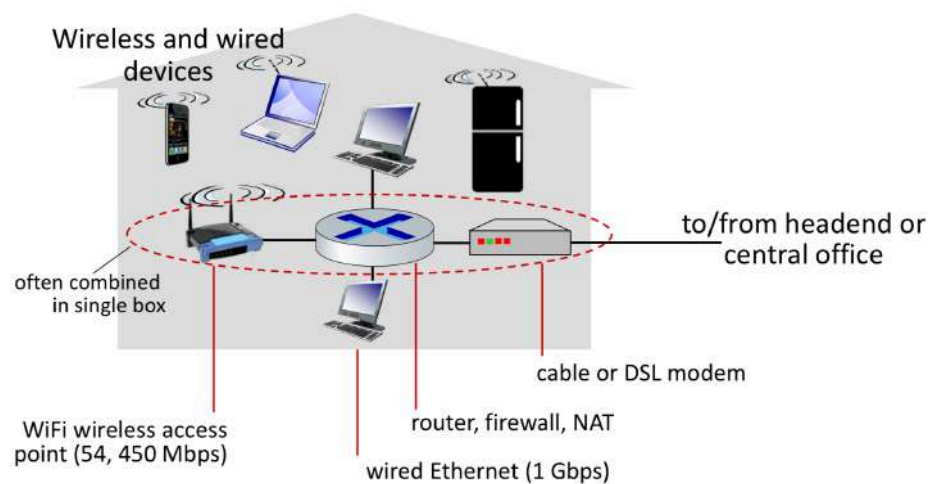
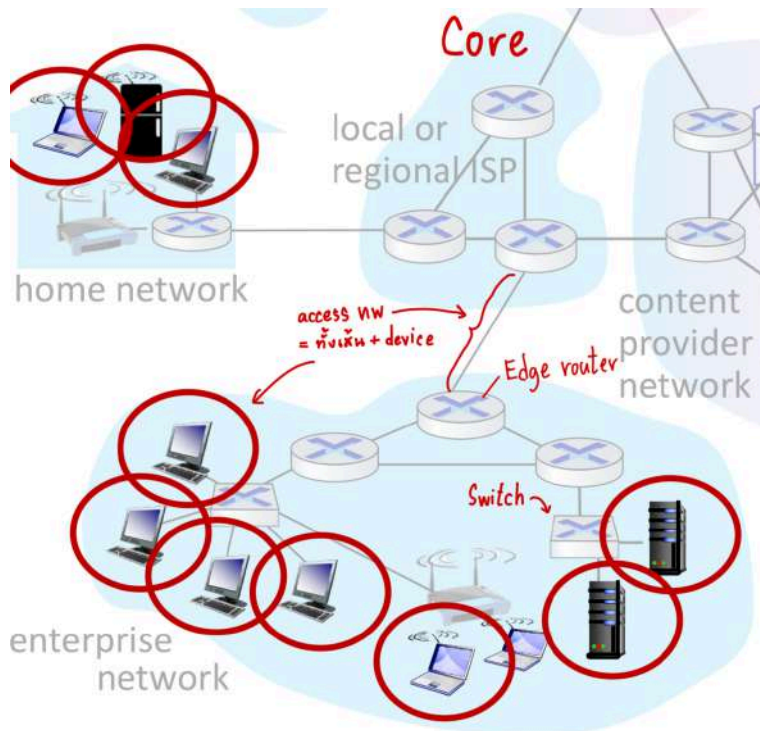


Access Networks > เหล่านี้เราแตะแต่พวก LAN

- Home network
- Mobile network (WiFi, 4G/5G)
- Enterprise network (School, Company)
- Datacenter network
- Content delivery network (Netflix, Amazon)

ส่วน network ที่อยู่ในเมฆ Internet เรียกว่า Core network

Network edge = ปลายสุดของ network คือ End systems => host ex. Clients & Servers
(edit: ตอนนี้อยู่ใจแล้วว่าคืออะไร เพราะจารย์บอกใน google classroom ว่าไม่ใช่แบบนี้ แต่ไม่ออกสอบ)



<< Home networks

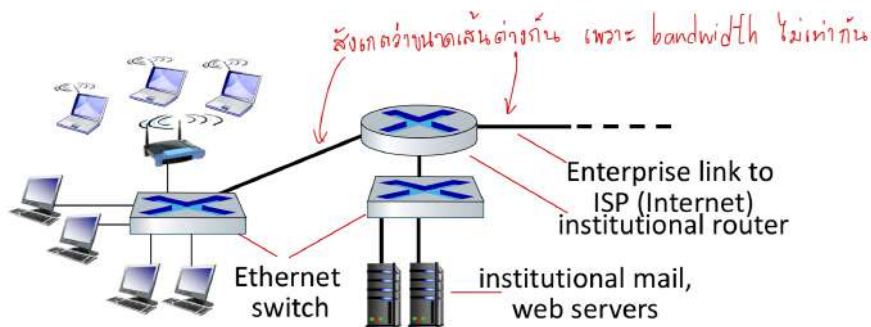
transmission rate == bandwidth

Wireless local area network (WLANs):

- อยู่ภายในหรือรอบอาคาร (~100 ft = ~30 m)
- 802.11b/g/n (WiFi): 11, 54, 450 Mbps

Wide-area cellular access networks:

- มีไว้ให้สัญญาณมือถือ (~10km)
- Bandwidth ~10Mbps
- 4G/5G cellular networks



<< Enterprise network
Ethernet: 100Mbps, 1 Gbps, 10Gbps
WiFi: 11, 54, 450 Mbps

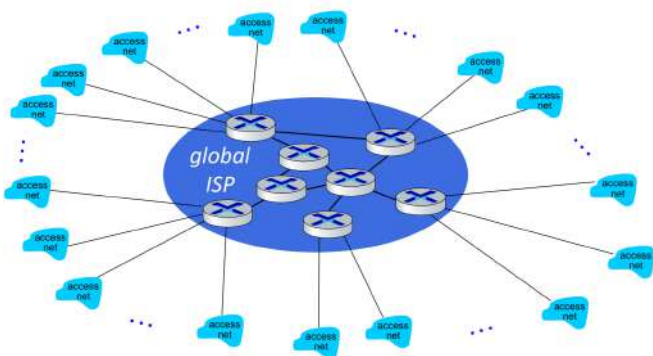
Data center network: high-bandwidth links (10 - 100 Gbps) connect เป็นร้อย server ด้วยกันกับ internet

Network core:

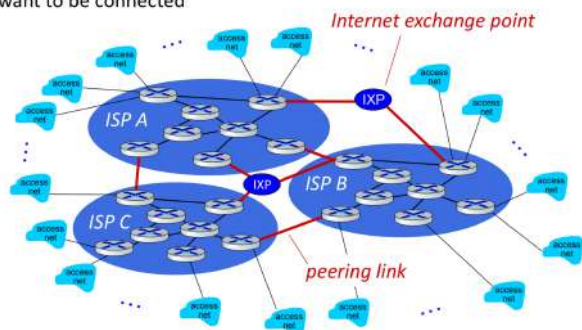
- Interconnected routers
- Network of networks

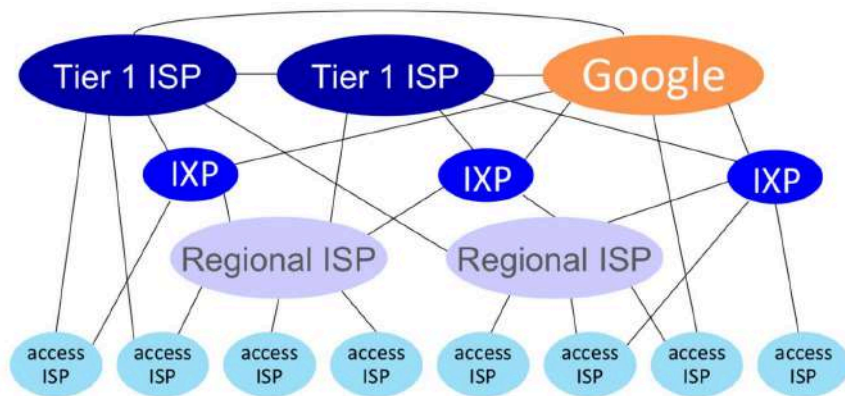
Network of networks: hosts connect กับ ISP แล้วที่นี้ด้วยความที่คนใช้เยอะมาก access ไปหา ISP ก็มากตาม ประมาณ $O(N^2)$ connections จึงต้องมี Global ISP เป็นตัวกลาง แต่ด้วยความที่มันรวมศูนย์เกินไป เลยมีหลายเจ้า และเวลาจะสื่อสารระหว่างกันใช้ Internet exchange point

*Option: connect each access ISP to one global transit ISP?
Customer and provider ISPs have economic agreement.*



But if one global ISP is viable business, there will be competitors who will want to be connected



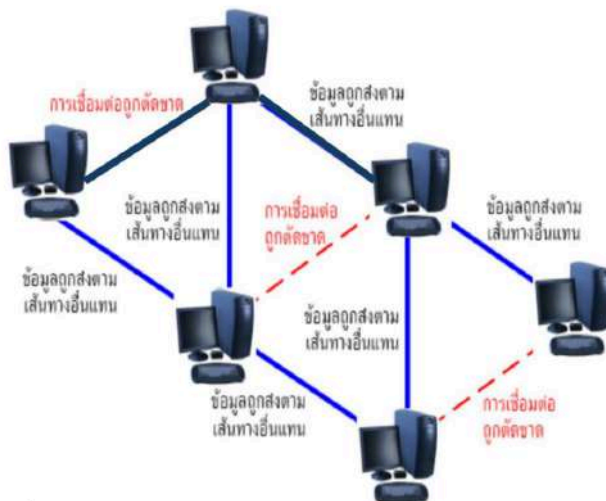


At “center”: small # of well-connected large networks

- “tier-1” commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- content provider networks (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

Fact: สมัยก่อนคนก็สังกัด Tier 1 แต่พอ Google ใหญ่ขึ้นก็เลยมาดูแลเอง

Packet Switch Network Advantage over Circuit Switch



Circuit Switch: สมมติเครื่อง A ต้องการคุยกับเครื่อง B เหมือนกับการโทรศัพท์ ก็คือจะจองเส้นไว้จนกว่าจะวางสาย บางทีก็จะเกิด Idle time เวลาที่ไม่ได้คุยอะไรแต่ยังค้างไว้

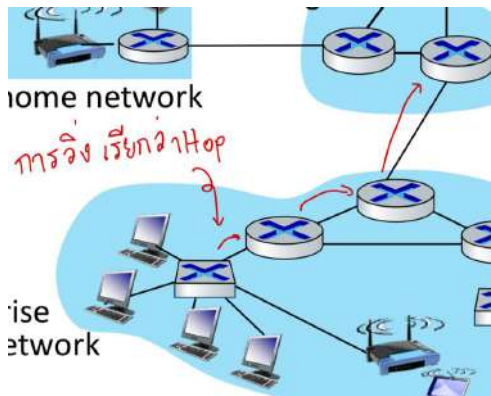
ข้อเสีย: เกิดการจองสาย, เวลา error สายก็จะตัดเลย

Packet Switch: เป็นการแบ่งการสนทนา แล้วส่งเป็น packet ทีละอัน ซึ่งการส่งแต่ละอัน เส้นทางจะไม่ซ้ำกัน โดยมีการสอน router ให้เลือกเส้นที่ดีที่สุดตอนนั้น (ใช้ Dijkstra ช่วย)

Ex. ถ้า traffic เยอะ มันก็จะเปลี่ยนเส้นทาง

Internet ใช้ packet switch โดยมี router เป็นคนเลือกเส้นทาง

Week 2: Protocol layer



เหตุผลที่ต้องเขียนเส้นเฉียงลงเพราะ
มีการเดินทางของข้อมูล >>>

2 reference models:

OSI: 7 layer model

Internet Protocol Stack (TCP/IP Stack): 5 layer model

Internet != World wide web

Application layer:

- เป็น layer เดียวที่ติดต่อกับ user โดยตรง
- SW App อย่าง web browsers และ email clients ฟังพา app layer เพื่อเริ่มการสื่อสารกับ servers
- มีหน้าที่รับผิดชอบ protocols และ การจัดการ data ที่ SW ใช้ในการ present แก่ user
- Ex. protocol => HTTP port 80, SMTP port 25, HTTPS port 443, DNS port 53, DHCP server 67 client 68

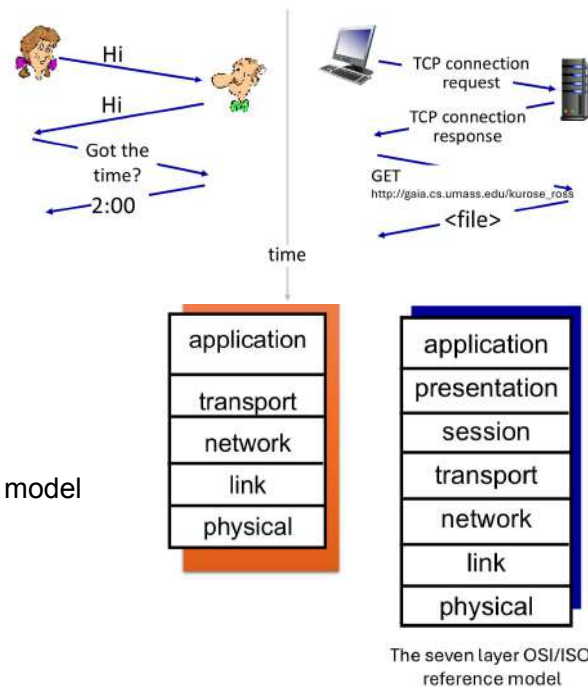
Presentation layer:

- มีหน้าที่เตรียม data เพื่อ app layer ไม่ว่าจะเป็น แปลภาษา (encoding), เข้ารหัสลับ และบีบอัดข้อมูล
- บางที 2 devices อาจจะคุยกันคนละ encoding methods ดังนั้น layer 6 จึงรับผิดชอบในการแปล data ที่เข้ามาให้กลายเป็น syntax ที่ app layer ของ device ที่รับนั้นเข้าใจได้
- ถ้าการสื่อสารมีการ encryption layer 6 ก็ต้องเข้ารหัสลับให้ผู้ส่ง รวมถึงถอดรหัสให้ผู้รับ
- นอกจากนี้ยังรับหน้าที่ในการบีบอัด data ที่รับจาก app layer ก่อนส่งไปให้ layer 5 เพื่อเพิ่มความเร็วและประสิทธิภาพ ด้วยการลดขนาดจำนวน data ที่ต้องส่ง

What's a protocol?

คือ สิ่งที่มีรูปแบบ ลำดับของข้อมูลที่ส่งและรับระหว่าง network entities และการกระทำที่ต้องทำในการส่ง message Ex. การ handshake

A human protocol and a computer network protocol:



The seven layer OSI/ISO reference model

Session layer:

- มีหน้าที่ในการ เปิด-ปิด การสื่อสารระหว่าง 2 devices ซึ่งเวลาระหว่างขณะที่เปิดและปิด เรียกว่า session
- Session layer ทำให้แน่ใจว่า session สามารถเปิดนานพอที่จะย้าย data ทั้งหมดที่จะส่ง และปิดทันทีเพื่อหลีกเลี่ยงการเสียทรัพยากร
- Session layer ยัง sync data transfer ด้วย checkpoints เช่น สมมติไฟล์ขนาด 100 MB กำลังถูกส่ง session layer อาจจะ set checkpoint ทุก ๆ 5 MB เพื่อในกรณีที่ disconnect or crash ที่ 52 MB ทำให้สามารถส่งต่ออีกครั้ง โดยที่ยังเหลืออีก 50 MB ที่ต้องส่ง ถ้าไม่มี checkpoints จะต้องส่งใหม่หมด

Transport layer:

- รับผิดชอบในการ end-to-end communication ระหว่าง 2 devices
- ซึ่งรวมถึงการนำ data จาก session layer แล้วนำมาแบ่งย่อย ๆ เรียกว่า segments ก่อนส่งไป layer 3
- ในฝั่งรับ มีหน้าที่ reassembling (ประกอบคืน) เหล่า segments ให้เป็น data ที่ session layer สามารถใช้ได้
- และยังดูแล flow control และ error control ด้วย
 - Flow control เป็นตัวกำหนดความเร็วการส่งที่ดีที่สุด เพื่อให้แน่ใจว่า sender ที่เน็ตเร็วจะไม่ส่งจนล้นให้กับ receiver ที่เน็ตช้า
 - ส่วน error control เป็นการเช็คข้อมูลนั้นส่งมาเสร็จสิ้น หากไม่ให้ส่งมาใหม่
- Ex. protocols => TCP, UDP

Network layer:

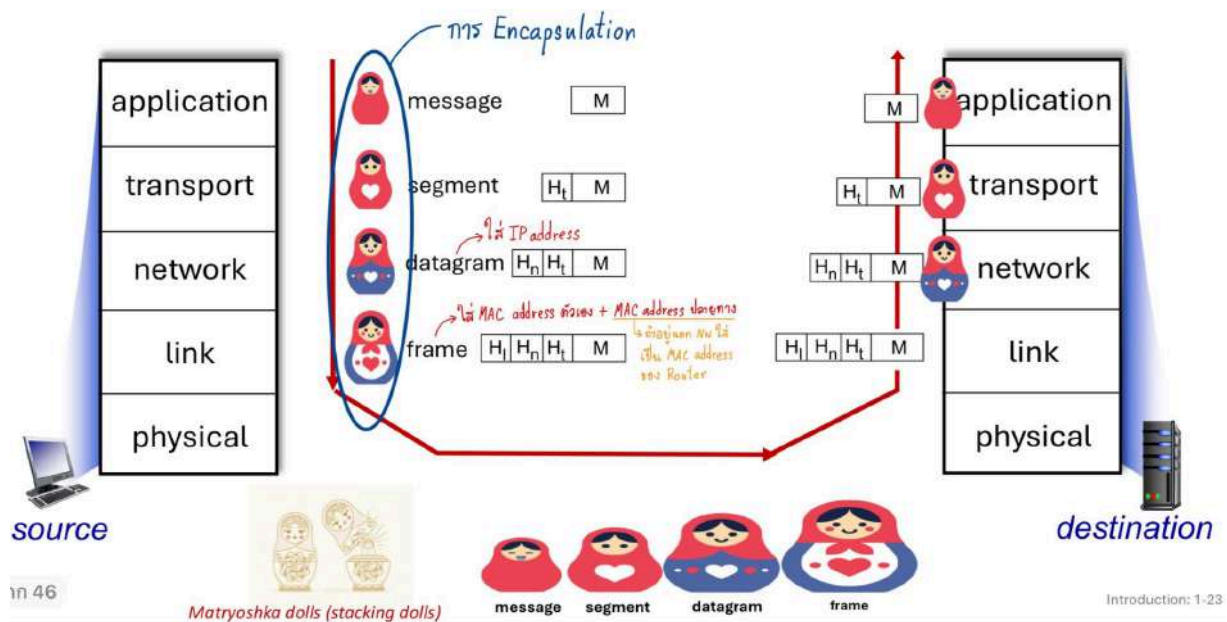
- มีหน้าที่อำนวยความสะดวกในการส่ง data ระหว่าง 2 network ที่ต่างกัน
- ถ้าส่งภายใน network เดียวกัน layer นี้ก็ไม่จำเป็น
- Network layer จะแตก segments ที่ได้จาก transport layer ให้เล็กลง เรียกว่า packets ในฝั่ง sender และประกอบคืนในฝั่ง receiver
- นอกจากนี้ layer นี้ยังหา physical path ที่ดีที่สุดในการไปถึงปลายทาง เรียกว่า routing
- Ex. protocols => IP, Internet Control Message Protocol (ICMP) => ใช้ ping เป็น message protocol , Internet Group Message Protocol (IGMP), IPsec suite และ routing protocols

Data Link layer:

- ไม่ได้ต่างจาก network layer ยกเว้นแค่ data link layer จะอำนวยความสะดวกในการส่ง data ระหว่าง 2 devices บน network เดียวกัน
- รับ packets มาจาก network layer แล้วแตกออกเป็นชิ้นเล็กๆ เรียกว่า frames
- นอกจากนี้ยังดูแลเรื่อง flow control และ error control ใน intra-network connection (ใน transport layer ดูแล flow control และ error control บน inter-network communications / คนละ network เท่านั้น) Ex. protocols > Ethernet, 802.11 (WiFi), PPP

Physical layer:

- Layer นี้จะมี physical equipment ที่ทำหน้าที่ transfer data เช่น สาย cables, switches
- เป็น layer ที่ data ถูก convert เป็น bit stream ก็คือ string ของ 1 และ 0
- Physical layer ของทั้ง 2 devices จะต้องตกลงการแปลงสัญญาณ (signal convention) เพื่อที่จะได้แยก 0 และ 1 ออกมาเหมือนกัน

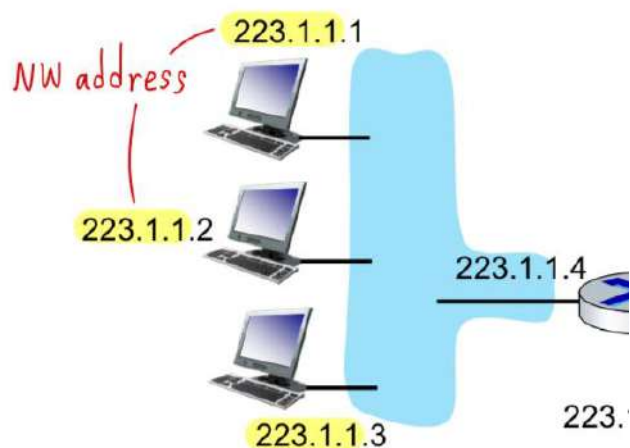


IP address: เป็นเลขขนาด 32-bit แต่ละตำแหน่งมีค่าได้ 0-255 มีไว้ใช้กำกับแต่ละ host หรือ router interface ซึ่งตามจริง computer จะเก็บไว้เป็นเลขฐานสอง

Interface: คือการเชื่อมต่อระหว่าง host/router กับ physical link

- Router โดยปกติมีหลาย interfaces
- Host มี 1 or 2 interfaces (Ex. wired Ethernet, wireless 802.11)

ใช้คำสั่ง ping [url/ip address] เพื่อดูว่า connect ได้ไหม



แล้ว host เอา IP address มาจากไหน?

- มี hard-coded ที่ฝังโดย sys admin ใน config file (/etc/rc.config in UNIX)
- DHCP : Dynamic Host Configuration Protocol ดึงมาโดยอัตโนมัติจาก server

Network security

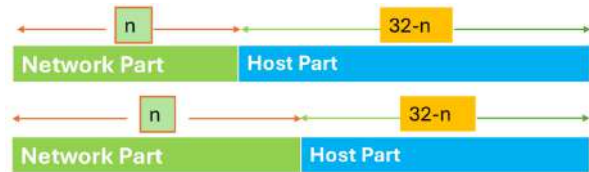
- Internet ไม่ได้ถูกออกแบบให้มี security มากนัก (สร้างโดยกระทรวงกลาโหม)
- ด้วยความที่ concept มั่นใจ เลยถูก adopt ไปใช้กับ business แต่ไม่ได้ถูกออกแบบใหม่
- มีปัญหาในการดักฟัง packet ได้เพราะทุกคนเชื่อมถึงกันภายใน network เรียกว่า packet sniffing
- ตัวอย่าง tool ที่ใช้คือ Wireshark

Week 3: IP Address

IP4 Component แบ่งได้ 2 ส่วน

- Network part => N bits
- Host part => 32-N bits

ถ้า network part n bit บางครั้งจะเขียนว่า /n



Subnet mask คือตัวแยกระหว่าง network part กับ host part

- โดยที่ใน 32 bit ค่าของ subnet mask จะให้ network part เป็น 1 ทั้งหมดทุก bit และส่วนที่เป็น host part เป็น 0 ทุก bit
- IP address 2 ค่าใดๆ เมื่อไป AND กับ subnet mask แล้วได้ค่าเหมือนกันแสดงว่าอยู่ network เดียวกัน

Host part คือค่าที่กำหนดให้แต่ละ host/เครื่อง โดยค่าจะไม่เหมือนกัน

- จำนวน bit ของ host part นำมาคำนวณได้ว่ามีหมายเลข IP address ได้ทั้งหมดกี่หมายเลข
- จำนวน IP ที่เป็นไปได้ทั้งหมด = $2^{(32-N)}$
- จำนวน IP ที่สามารถใช้ได้ใน NW นั้นๆ = $2^{(32-N)} - 2$ (ลบ 2 เพราะให้ตัวแรกเป็น NW address และตัวท้ายเป็น Broadcast address)

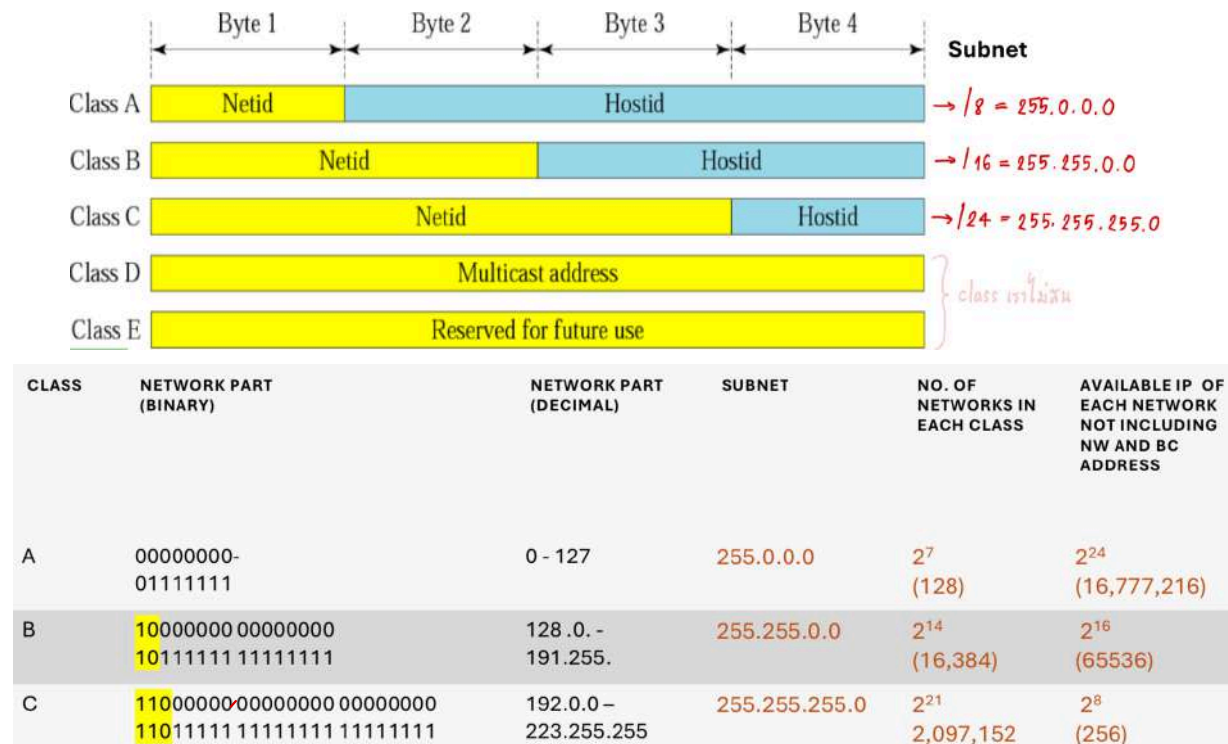
Network address = IP address AND Subnet mask (AND คือ ต้อง 1 ทั้งคู่ ถึงจะ = 1) เป็น IP แรกของ network

223	1	3	27	
11011111	00000001	00000011	00011011	IP Address
11111111	11111111	11111111	00000000	Subnetmask
11011111	00000001	00000011	00000000	
223	1	3	0	

Broadcast address = Network address แต่ flip bit ส่วน host part เป็น 1 ทั้งหมด เป็น IP สุดท้ายของ network

IP address ของ Host	223	1	3	27
ฐานสิบ				
IP address ของ Host	11011111	00000001	00000011	00011011
ฐานสอง				
Subnet (ฐานสอง)	11111111	11111111	11111111	00000000
Network address	11011111	00000001	00000011	00000000
ฐานสอง				
Network address	223	1	3	0
ฐานสิบ				
Broadcast address	11011111	00000001	00000011	11111111
ฐานสอง				
Broadcast address	223	1	3	255
ฐานสิบ				

Classful addressing: เป็นการแบ่งขนาด Subnet mask ตามจำนวน byte ซึ่งในสมัยก่อนชอบใช้ โดยแบ่ง



แต่ว่ามันมีปัญหา คือ จำนวน NW ID และ Host ID มันไม่ make sense เช่น Class A มี nw ได้ $2^7 = 64$ กับมี host ได้ $2^{24} = 16,777,216$ เครื่อง แต่ Class C มี nw ได้ $2^{21} = 2,097,152$ กับมี host ได้ $2^8 = 256$ เครื่อง ซึ่งมันมาก/น้อยเกินไป ดังนั้นเลยเกิด Classless InterDomain Routing (CIDR) ที่มี format เป็น a.b.c.d/x โดยที่ x คือเลข bits ของสัดส่วน subnet

โดย router จะดูแค่ IP address และ Network address ในการเลือกเราเหมือนรหัสไปรษณีย์คือ ดูเป็นโซน

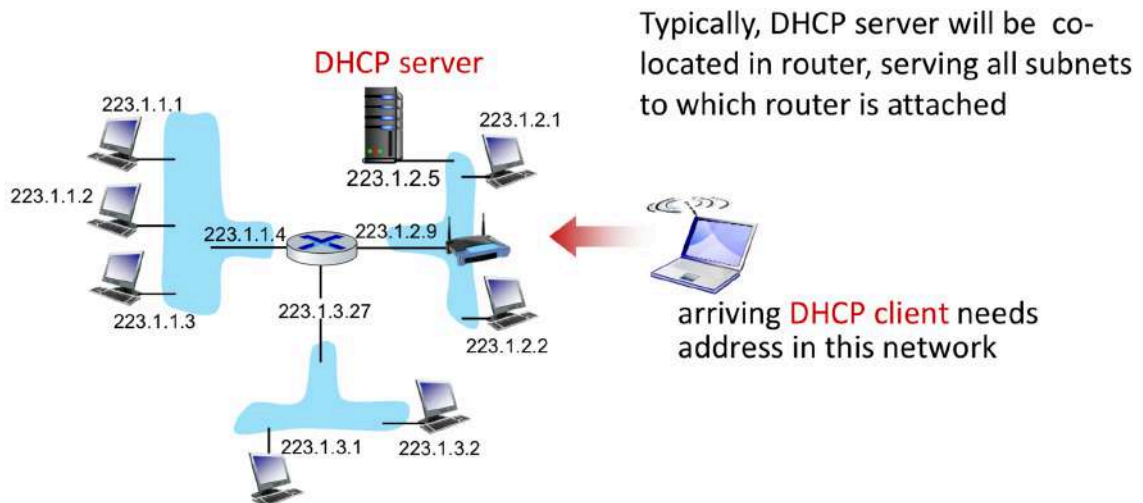
Default gateway: เอามาจากหมายเลข IP ที่ available 1 อันในกรณีที่ต่อ network ข้าม internet ให้ router ซึ่งจริงๆ เป็นหมายเลขอะไรก็ได้แต่มักจะ assign เป็นหมายเลขแรกหลัง nw address หรือ หมายเลขสุดท้ายก่อน broadcast address

DHCP นอกจากช่วย assign IP address ให้กับเครื่องคอมแล้ว ยังช่วยบริหารจัดการ IP ด้วย

- Ex. สมมติมีคอม 300 เครื่อง แต่มี available IP = 253 นั้นหมายความว่ามันไม่พอ แต่ว่าเราสามารถ reuse ได้ ในกรณีคนใช้เครื่องคอมไม่พร้อมกัน โดยไม่ต้องไปขอจด IP เพิ่ม
- แสดงว่าเบอร์ IP address ไม่ได้ fix กับเครื่อง สามารถ assign ไปให้เครื่องอื่นได้
- Lease time คือระยะเวลาที่บอกว่าสามารถใช้หมายเลข IP นี้ได้นานเท่าไร โดยที่หากเครื่องนั้น ๆ ไม่มี interaction ใด ๆ อีกเลย หลังจาก lease time มันก็จะตัด IP นี้ออก แล้วให้คนที่เข้ามาใหม่ใช้

How DHCP work?: ในองค์กรต้องมี DHCP server 1 เครื่อง โดยจะอยู่ใน nw เดียวกับเราหรือไม่ก็ได้ ขอแค่ให้ต่อกันได้ ก่อนออก internet

DHCP client-server scenario



ดังนั้นหากมี com เครื่องใหม่เข้ามาใน nw ตัวเครื่องก็จะป่าวประกาศเพื่อที่ขอ IP address
DHCP ก็จะ assign => IP address, Subnet mask, Default gateway, DNS address, Lease time

Overview

Step 1: DHCP discover = ทำการ Broadcast ตะโกนถามทุกคนใน nw เพื่อหา DHCP server

- หากเข้ามาแล้วยังไม่มีหมายเลข IP ให้ใช้เป็น 0 ทั้งหมด (0.0.0.0) แล้วถามไปยัง broadcast address ที่ซึ่งมีค่าเป็น 1 ทั้งหมด => ถามทุกคนในทุก nw ที่ต่อได้ก่อนออก internet
- โดยมี fixed port ไว้เฉพาะ ถ้าคุยกับ DHCP server จะ UDP port 67 ส่วน client port 68
- ด้วยความที่ยังไม่มีหมายเลข IP เลยต้องมีค่า reference ID ที่เวลาตอบกลับมา เครื่องที่ถามจะได้รู้ว่าตอบกลับตัวเอง ก็คือ transaction ID ซึ่งเป็น random number อะไรก็ได้

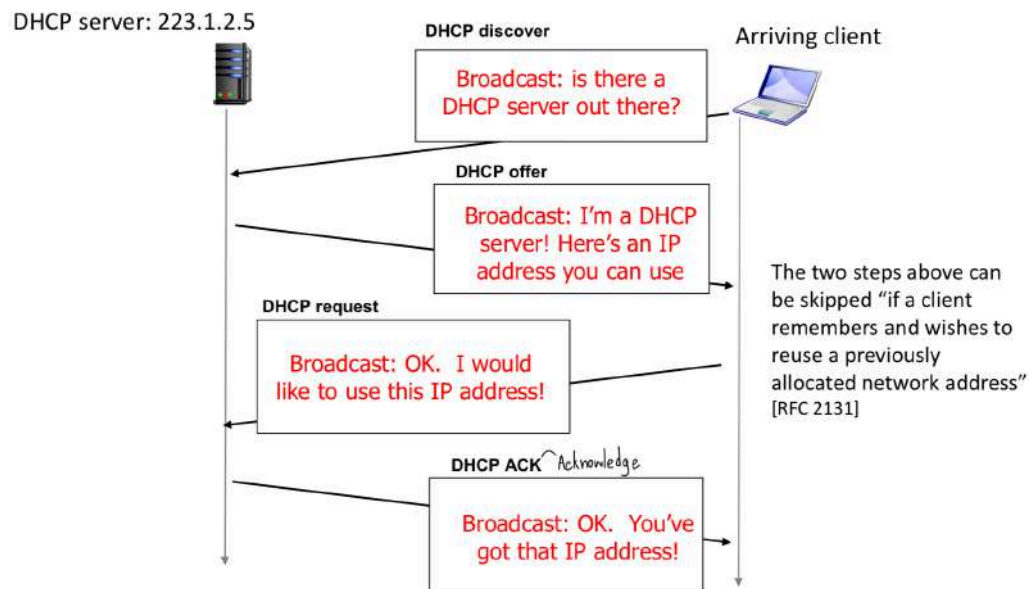
Step 2: DHCP offer = DHCP server เปิดอยู่ตลอด เมื่อมีเครื่องขอมาก็จะ offer IP address ให้

- ตัว host นั้นสามารถเลือกได้ว่าจะรับ IP address จาก DHCP server อันไหน ซึ่งปกติก็เลือกอันใกล้ที่สุด

Step 3: DHCP request = host เลือกรับ IP address

Step 4: DHCP ack = DHCP server ก็จะทำการส่ง IP address ดังกล่าวให้ และ assign

DHCP client-server scenario



Fact: อย่าง router บ้าน or mobile hotspot มันจะมี DHCP server built-in อยู่ในเครื่องอยู่แล้ว

Subnetting

- เป็นการแบ่ง network ใหญ่ออกเป็น network ย่อยๆ ได้โดย

1. ให้จำนวน network ที่ต้องการเป็น x
2. หาค่า $\log_2(x)$ บัดขึ้นได้เป็น y = จำนวน bits ที่ขอยืมจาก host part มาเป็น sub network part
3. Subnet mask ใหม่จะเป็น $/x+y$ และ host part ของแต่ละ subnet ใหม่จะเหลือ $32-n-y$ bits
4. จำนวน IP ทั้งหมด (รวม nw address, broadcast address ของ subnet ใหม่) = $2^{(32-n-y)}$

$(2-n-y)$



องค์กรจะจัด IP address ได้อย่างไร?

- จัดเสียดังกับ ICANN (Internet Corporation for Assigned Names and Numbers) ซึ่งก็จะแบ่งเป็น 5 โชนเรียกว่า regional registries (RRs)
- RRs เป็นคนดูแล และ allocate (จัดสรร) ว่าโชนนี้มี IP address available อะไรบ้าง + ดูแล DNS ด้วย
- ICANN ได้ assign IPv4 address หมดไปแล้วเมื่อปี 2011

NAT: network address translation << เข้ามาช่วย IPv4 ที่หมดไปแล้ว

- ใช้ concept ของ private IP ก็คือทุก devices ใน local nw จะใช้ IPv4 address ร่วมกันหนึ่งตัว
- ส่วน IP ที่ได้รับการ assign มา => public IP address (คนด้านนอกเห็นเราแบบนี้)
- เมื่อก่อน Private IP จะมี format เป็น 10.x.y.z/8 or 172.16.x.y/12 or 192.168.x.y/16 ก็คือมีตาม class แต่ว่าปัจจุบันจะ slash อะไรก็ได้

Week 4: Application Layer Protocol Concept and HTTP

Network application = Application ที่มีการเชื่อมต่อ internet เพื่อใช้งาน

- นั้นหมายความว่า Application ที่ run บนเครื่องอย่างเดียว = Application เฉยๆ ไม่ใช่ nw app
- Nw app Ex. Social networking, Web, Text messaging, Multi-user nw games, streaming stored video (YT, Netflix), Voice over IP (Skype), Real-time video conferencing (Zoom)

Architecture ของ Network Application

1. Client-server architecture

Server:

- เป็น host ที่เปิดตลอดเวลา
- IP address ถาวร
- มักอยู่ใน data centers โดยสามารถ scaling ได้
- จะไม่ติดต่อหา client ก่อนเลย (ยกเว้นใช้ push noti ถ้าไม่ใช่แสดงว่าเจอ attack)

Client:

- ติดต่อสื่อสารกับ server
- มีการ connect แบบเข้า ๆ ออก ๆ ไม่ต้องเปิดตลอดเวลา
- อาจมี dynamic IP address
- ไม่สื่อสารกับ client ด้วยตัวเอง

- Ex. HTTP, IMAP, FTP

2. Peer-peer architecture

- ไม่มี server ที่เปิดตลอด
- เครื่องสามารถเป็นได้ทั้ง client & server ทั้งให้และรับ
- เข้าออก network เมื่อไหร่ก็ได้ โดยที่มีการเปลี่ยน IP address ด้วยทำให้ complex management
- Ex. P2P file sharing (BitTorrent) = Download ไฟล์คนอื่นได้ คนอื่นก็มา download ไฟล์เราได้ โดยที่จะมี protocol ที่หากรับอย่างเดียวไม่ให้เลย จะตัดออกจาก nw

ส่วนใหญ่แล้ว Application เป็นแบบ client-server

Processes communicating — App layer

process: program ที่ running ภายใน host

- ภายใน host เดียวกัน 2 processes สื่อสารโดยใช้ inter-process communication (กำหนดโดย OS)
- Processes ใน host ที่ต่างกันจะสื่อสารโดยแลกเปลี่ยน messages

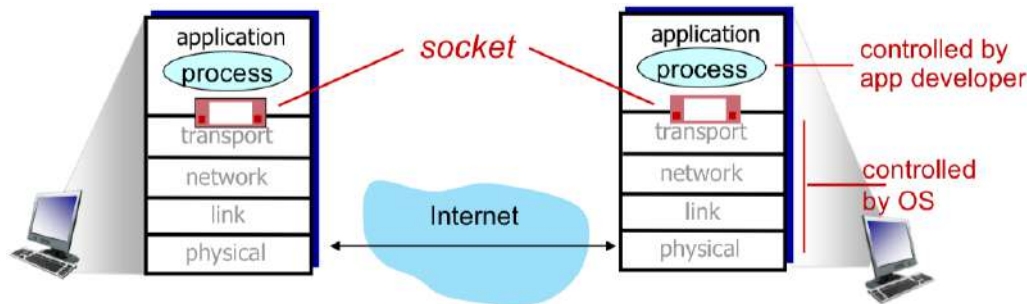
client process: process ที่ริเริ่มการสื่อสาร

server process: process ที่รอการติดต่อ

Note: Applications ที่เป็น P2P จะมี client processes และ server processes

Sockets

- process นั้นจะ ส่ง/รับ messages ไปหา/จาก ตัว socket ของมัน
- Socket เปรียบเสมือนประตู
 - Sending process จะผลัก message ออกจากประตู
 - Sending process พึ่งพา transport infrastructure ทางฝั่งอื่นของประตู ในการส่ง message ไปหา socket ที่ receiving process
 - ก็คือแต่ละฝั่งจะมี socket คนละหนึ่งอัน



Addressing process

- ที่นี้ในการรับ message นั้น process ก็ต้องมี identifier เพื่อรู้ว่าเป็น socket อะไร เครื่องไหน
- Identifier ประกอบด้วย IP address และ port numbers
- Ex. port number ว่าเรากำลังคุยกับใคร => HTTP server: TCP 80, mail server: TCP 25, DHCP server: UDP 67, HTTPS server: TCP 443

ยกตัวอย่างจะส่ง HTTP message ไปหา gaia.cs.umass.edu web server:

- IP address: 128.119.245.12
- port number: 80

Note: port number มาตรฐานของ server จะไม่เกิน 1024 หาก port number เลขเยอะๆ มักจะเป็น protocol ที่ไม่ได้มาตรฐาน หรือมักจะเป็นเบอร์ของ client ที่เปลี่ยนตลอดเวลา เพราะ server ไม่เปลี่ยน

ออกแบบ Application-layer protocol ต้องมีอะไรบ้าง

- ประเภทของ message ที่แลกเปลี่ยน เช่น request, response
- Message syntax: มี field อะไรบ้างใน message & field ต่างๆ ถูกแยกยังไง เช่น message นี้ให้ client ทำคำสั่ง... 1.ประธานทำอะไร ก็ต้องมีบอกกี่ bits
- Message semantics: ใช้อธิบายความหมายของข้อมูลใน field ต่าง ๆ
- Rules ใช้กำหนดว่า process จะ send & respond กับ message เมื่อไหร่ ยังไง
- Open protocols: เป็น protocol ที่เรารู้มาตรฐาน เช่น ที่ wireshark สามารถเอามาแล้วแปลความหมายได้ ถูก define ใน Request for Comments (RFCs)
- RFCs ถูกดูแลโดย Internet Engineering Task Force (IETF) ทุก protocol จะต้องเขียนรูปแบบไปแจ้ง IETF แล้วเขาจะมี working group เพื่อดูว่า protocol นี้ใช้งานได้หรือไม่ และใช้ทำอะไร ถ้ามี update ก็ต้องแจ้งขึ้นไป
- ทุก protocol จะมี RFC number (Ex. HTTP > RFC7230)
- มีบาง protocol ที่ทำขึ้นเอง เรียกว่า proprietary protocols (Ex. Skype, Zoom) ไม่บอกว่า protocol ตัวเองคุยยังไง

แล้วรู้ได้ยังไงว่า app จะเลือก UDP or TCP? ต้องพิจารณาดังนี้

1. Data integrity = ยอมให้ data เกิดการ lost ชัก bit ได้ไหม ถ้าไม่ยอม = มี Data integrity
 - ไม่ยอม > app จำพวก file transfer, web transactions, mail ต้องการให้ย้าย data ครบ 100% ไม่ผิดเพี้ยน
 - ยอม > app อื่นๆ พวก audio สามารถมี loss ได้บ้าง
2. Timing = ต้องการเวลา delay ต่ำไหม /อยากได้ real-time รีบเปล่า
3. Throughput = อัตราการส่งข้อมูลผ่านเครือข่ายหรือระบบในช่วงเวลาหนึ่ง
 - บาง app พวก multimedia ต้องการ throughput ขั้นต่ำจำนวนหนึ่ง ถึงจะสามารถใช้งาน app ได้อย่างมีประสิทธิภาพ
 - บาง app ไม่แคร์ ใช้ได้มากก็ใช้มาก น้อยก็ใช้น้อย จะเรียกว่า "Elastic apps"
4. Security = ต้องการความปลอดภัยไหม? เพราะทุกอย่างที่ส่งใน internet ไม่มีอะไรปลอดภัย



Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video: 10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

TCP Vs UDP

TCP service:

- reliable transport = process ส่งและรับ ไม่มี data loss
- flow control: ป้องกันไม่ให้ receiver ได้รับข้อมูลมากเกินไปจนจัดการไม่ทัน
- congestion control: จัดการกับการแออัดในเครือข่าย โดยควบคุมปริมาณข้อมูลที่ส่ง เพื่อหลีกเลี่ยงไม่ให้เครือข่ายเกิดการแออัดจนประสิทธิภาพลดลง
- connection-oriented: ต้องมีการสร้างและรักษาการเชื่อมต่อระหว่าง sender และ receiver ก่อนที่จะเริ่มส่งข้อมูล
- แต่ว่าไม่มี > timing, minimum throughput guarantee และ security

UDP service:

- unreliable data transfer: ส่งไปแล้ว รับแคไหนก็ได้แค่นั้น
- ไม่มี flow control, congestion control, timing, throughput guarantee, security or connection setup

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Note: ด้วยความที่ internet bandwidth นั้นเยอะขึ้นมากแล้ว google เห็นว่าคนใช้ internet เยอะเลยอยากทำ streaming บน HTTP เลยสร้างขึ้น DASH ขึ้นมา

Securing TCP/UDP

ยังไงก็ตาม ไม่ว่าจะ TCP or UDP ไม่มี security อะไรเลยใน HTTP 1.0, 1.1 เรียกว่า Vanilla TCP & UDP

- ไม่มี encryption ส่ง password ไป เห็นเป็น plaintext ได้เลย

เลยมีการพัฒนา security บน TCP ก่อน เพราะเชื่อถือว่าข้อมูลมาเยอะ มาครบก่อน เรียกว่า

Transport Layer Security (TLS)

- มีการ encrypt TCP connections
- อยู่ครั้งแรกใน HTTP 2.0
- มี data integrity
- มี end-point authentication

ทว่ามี Google engineer ที่ทนไม่ได้ในการที่ต้องใช้ TCP เพื่อไม่ให้ data loss เลยสร้าง UDP ที่ไม่ loss ขึ้น และเรียกว่า TCP จากนั้นเขียน application ส่วนจัดการ control data loss ทั้งหมด เรียกว่า

Quick UDP Internet Connections (QUIC)

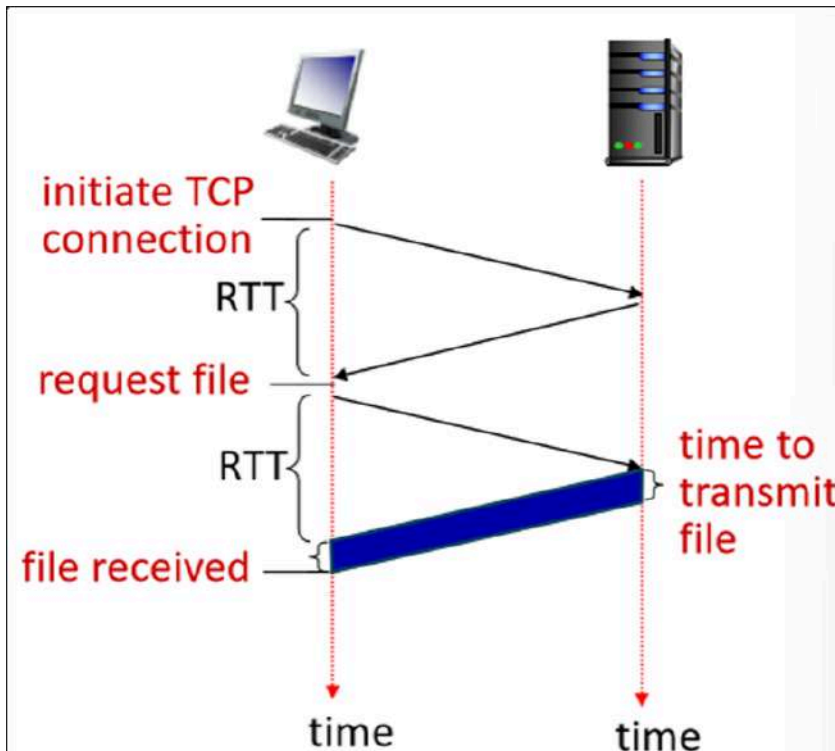
- มี security
- อยู่ใน HTTP 3.0

Web and HTTP

type of HTTP

- Non persistent - รับส่งข้อมูลได้ทีละ Object ในคราวเดียว
- Persistent - รับส่งข้อมูลได้ทีละหลายๆ Object ในคราวเดียว

RTT -> เวลาไปกลับของข้อมูลระหว่างทั้งสองฝั่ง



ให้ Transmission Time ของ Object1 object2 object3 ... object10 เป็น $t_1, t_2, t_3, \dots, t_{10}$ ตามลำดับ

• กรณี Non- Persistent HTTP

$$(2RTT \times 10) + T_1 + T_2 + \dots + T_{10}$$

• กรณี Persistent HTTP

$RTT + 10RTT + T_1 + T_2 + \dots$

~~$2RTT + T_1 + T_2 + \dots + T_{10}$~~

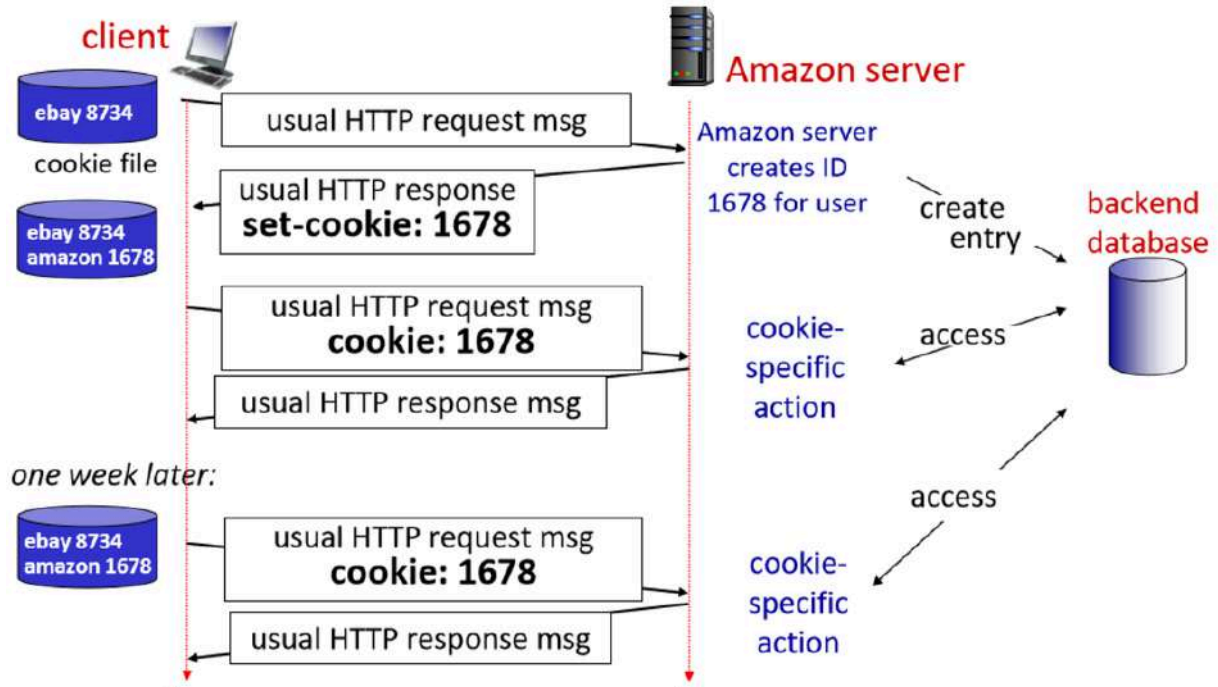
$RTT + (RTT + t_1) + \dots$

ส่งครั้งเดียว
หลายไฟล์

HTTP โดยลำพังมันไม่มีความจำ (stateless) เพราะจันทันทีเลยมีอะไรมาแก้ทาง นั่นก็คือ

COOKIE

ไฟล์เล็กๆ ที่เซิร์ฟเวอร์สร้างให้แล้วส่งให้ผู้ใช้เก็บไว้ ข้างในอาจจะบันทึกข้อมูลต่างๆที่ช่วยให้เซิร์ฟเวอร์จำได้ว่าเราคือใคร ชอบอะไร ยังไง จะได้ไม่ต้องถามซ้ำ น่ารำคาญถูกไหม



แน่นอนว่า มันก็มีพวก 3rd party cookies

คือแบบเซฟตรง เซฟที่เราต้องการจะไปขอหน้าเว็บมา มันก็ให้คุกกี้ แล้วหน้าเว็บมันก็ตั้งโปรแกรมให้ไปเรียกเว็บ

โฆษณาที่เก็บคุกกี้เหมือนกันอีก กรรม ที่นี้คุกกี้ของทั้งสองเว็บก็ถูกส่งมาเก็บมาไว้ที่เรา หวานหมูเว็บโฆษณาเลย

เว็บโฆษณา มันก็ดูว่าเราเข้าเว็บ(ที่เราต้องการเข้าจริงๆ) ว่ามีเนื้อหาประมาณไหน ละมันก็จะเก็บไว้ใน คุ ก ก็

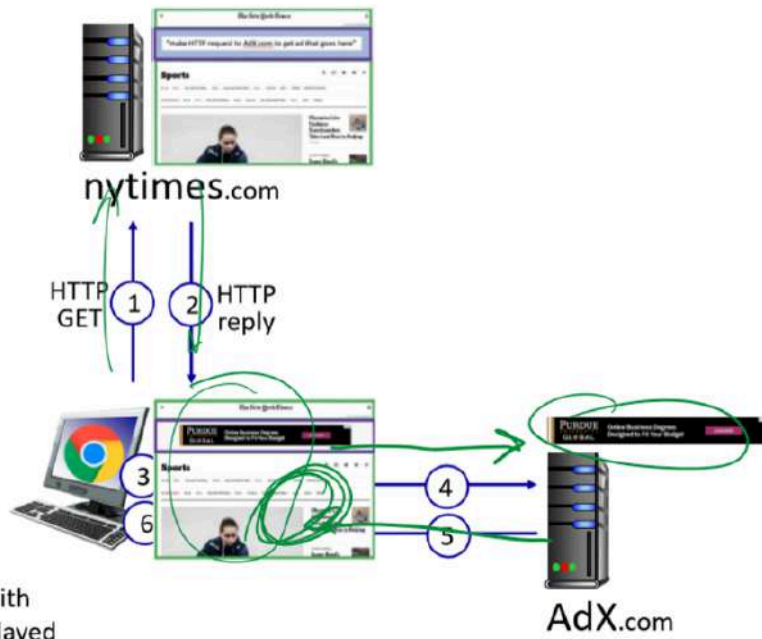
ละพอเราเข้าเว็บอื่นที่มีโฆษณาจากเว็บเจ้าเดียวกัน มันก็จะเอา คุ ก ก็ ของ มัน ที่เก็บไว้กับเรา มาแสดงโฆษณา

แบบเดียวกันให้เราเห็น (นึกถึง LAZADA)

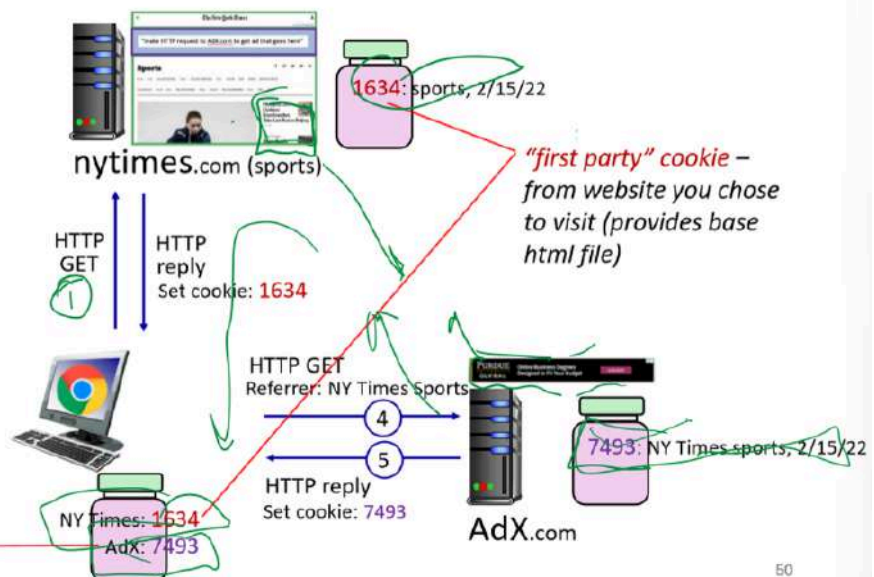
1 GET base html file
2 from nytimes.com

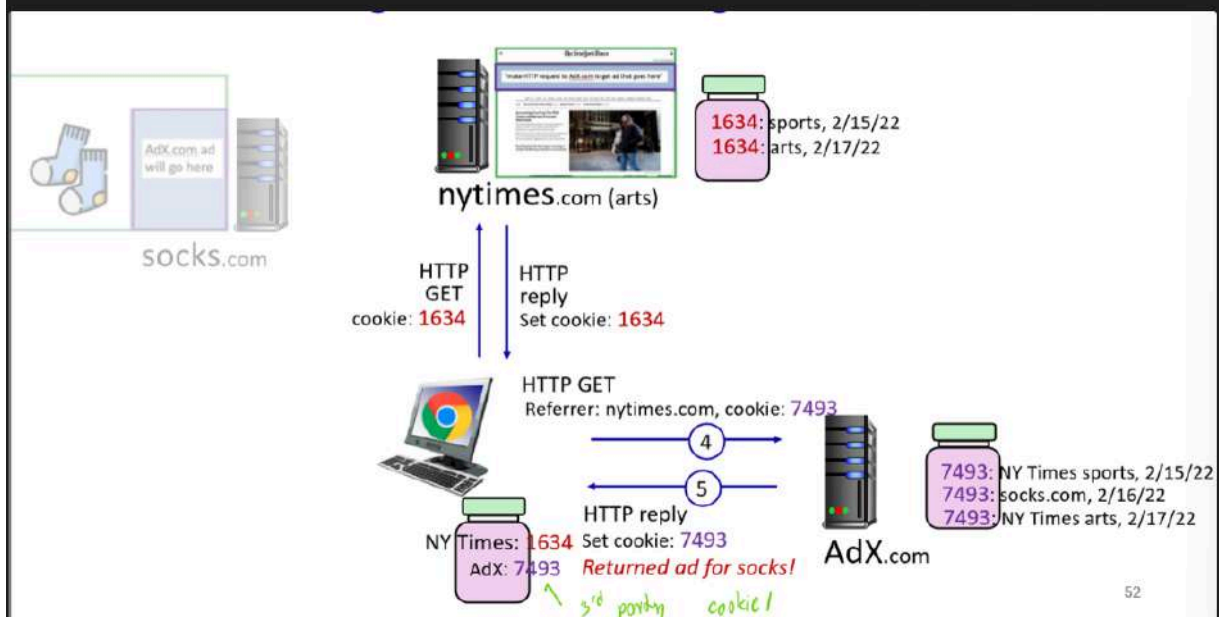
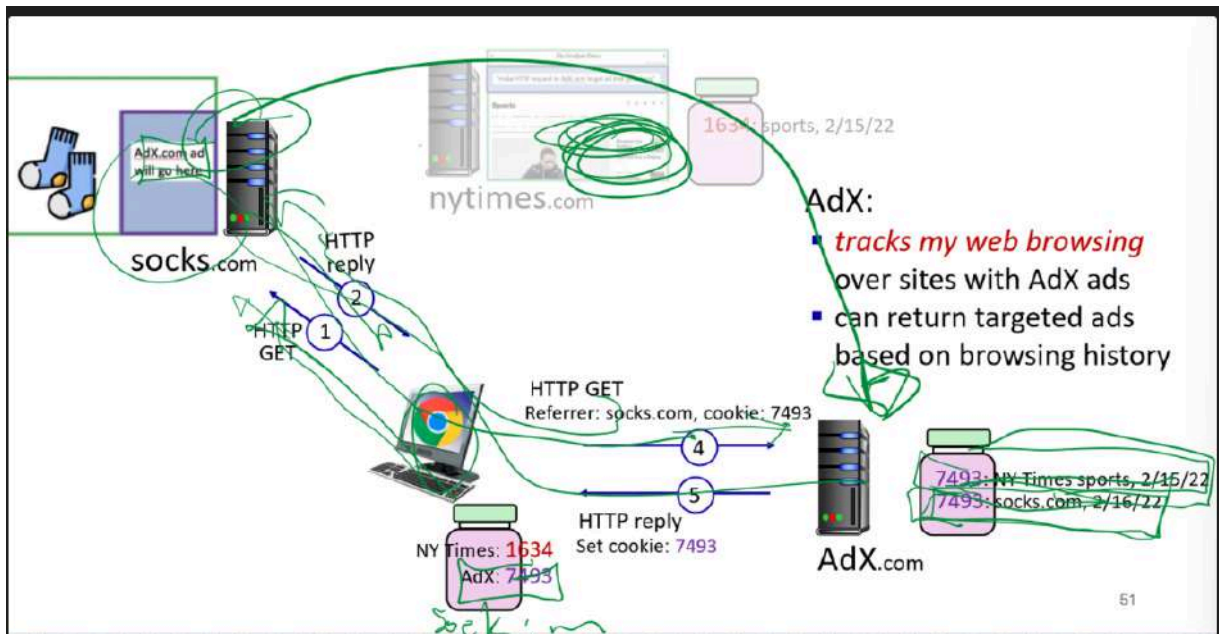
4 fetch ad from
5 AdX.com

7 display composed
page



*"third party" cookie –
from website you did not
choose to visit*





เพราะ 3rd party cookie นี้เอง เลยทำให้

เกิดการรุกร้าความเป็นส่วนตัวเกิดขึ้น

ซึ่ง **GDPR** ไม่ยอม

เลยออกกฎว่าจะมีการใช้คุกกี้ ต้องขออนุญาตผู้ใช้(ที่หมายถึงพวกเรา)ก่อน และอธิบายรายละเอียดในการใช้ให้ชัดเจน พร้อมกับตัวเลือกในการรับ คุกกี้ ที่สามารถรับได้ และไม่รับ

Web caches (Proxy servers)

บางทีการขอให้เซิร์ฟเวอร์ส่งหน้าเว็บมาให้ ซ้ำๆ เดิมๆ มันก็น่าเบื่อ เซิร์ฟก็จะบอก จะส่งมาอะไรกันหน้า ให้มันซ้ำซ้ำ เสียเวลาเชื่อมต่อ เสียเวลา Handle request เก็บ Cache ไว้เองซะสิจะได้ไม่ส่งมา

แต่ถ้าฝั่งผู้ใช้ทำ Request มาอีกแน่นอน เลยมีการ Conditional GET เกิดขึ้น

ให้ผู้ใช้ส่งมาว่า เข้าดูล่าสุดเมื่อไหร่ ถ้าเซิร์ฟยังไม่มีการเปลี่ยนแปลงเลยตั้งแต่ตอนนั้น

(return ERROR 304: Not modified) แต่ถ้าหน้าเว็บเซิร์ฟเรามีการเปลี่ยนแปลง จะส่งหน้าเว็บใหม่ไปให้นะ

HTTP requests

- GET
- POST
- PUT
- DELETE
- HEAD → get but returns headers only

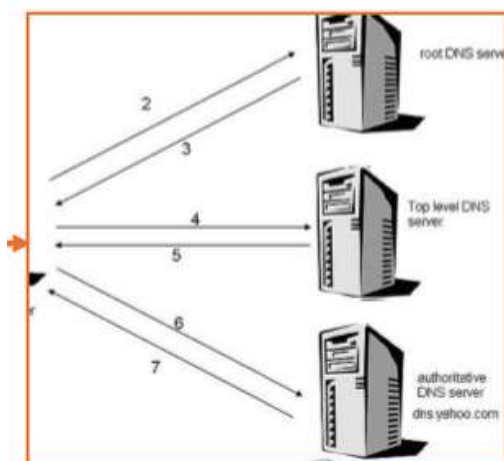
HTTP response code

- 200 → OK
- 301 → Moved permanently
- 304 → not modified
- 400 → bad request (หน้าบ้านพัง)
- 500 → server error (หลังบ้านพัง)
- 505 → HTTP ver unsupported

Socket Programming

มีไว้เพื่อให้แอปเซิร์ฟและผู้ใช้งานสามารถใช้ socket ในการสื่อสารกัน สามารถใช้ได้ทั้ง TCP and UDP

Week 5: Application Protocol Ep.2 DNS -> Domain Name System



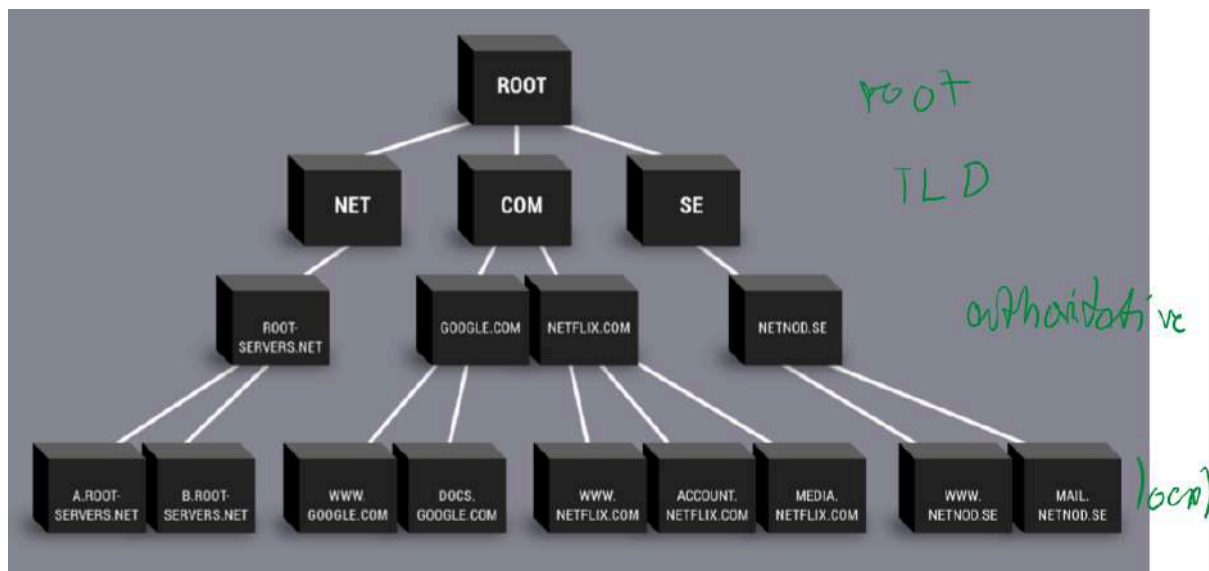
DNS เกิดมาเพื่อเป็นชื่อเรียกแทนเลขไอพีเซิร์ฟเวอร์ ซึ่งจะเก็บข้อมูลคู่ชื่อกับเลขไว้ในDNS Server(ต่อไปนี้จะเรียก DNSS)

โครงสร้าง DNSS จะกระจายๆ ดังรูป
กระจายทำไม?

- ลดความเสี่ยงเวลาเซิร์ฟใดล่ม
- กระจายกว้าง client ย่อมเข้าถึง DNSS ได้เร็วกว่า
- การบำรุงรักษาที่ถูกลง

Distributed database storing resource records: **RR**

DNS is distributed like this:



Root -> จุดสูงสุดของ DNSS มีแค่ 13 ชื่อเท่านั้น ก็คือ A...M.root-servers.net บริหารจัดการโดย ICANN
Top Level Domain (TLD) -> พวกโดเมนพ่วงท้ายทั้งหลาย registry of Authoritative servers
Authoritative server -> โดเมนหลักที่จดทะเบียนโดยองค์กรต่างๆ (และองค์กรเหล่านั้นสามารถบริหารจัดการได้ด้วยตัวเอง)

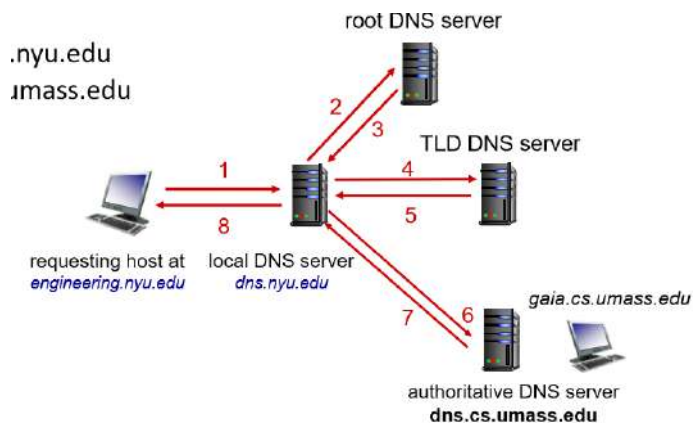
Local server -> ย่อยๆของประเภท Author server
เช็คได้ด้วย

url DNS IP

- `nslookup somedo.main <optional DNS server IP specifying>`
- `nslookup -type=NS somedo.main` (check authoritative servers) <optional DNS server IP specifying>
- `nslookup -type=MX somedo.main` (check mail servers) <optional DNS server IP specifying>
- `nslookup -type=CNAME somedo.main` (check Canonical (real) name) <optional DNS server IP specifying>

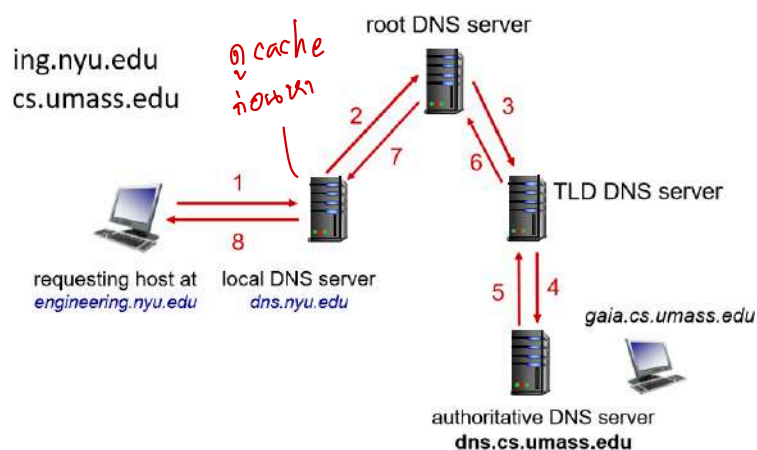
DNS query methods

- Iterative



local server รับโหลดไปได้เรื่อยๆ เลยไม่ค่อย
ได้รับความนิยมเท่าอีกแบบ

- Recursive



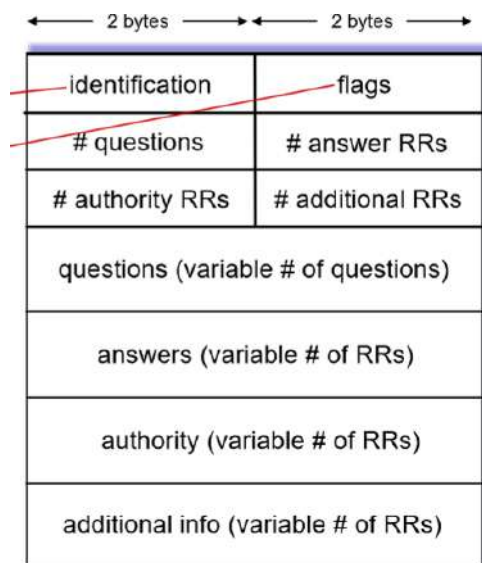
นิยมสุด ๆ

Caching DNS info

คือการบันทึกการแมพชื่อเข้ากับไอพี ทำให้ไม่ต้องไปขอเลขไอพีซ้ำ ทำให้โหลดเว็บได้เร็วขึ้น (ซึ่งจุดนี้เองก็เมื่อเว็บถูกย้ายที่อยู่ใหม่ เราก็ไม่รู้เรื่อง)

DNS protocol messages

both query and reply msg have same format



Header

Identification -> ว่าเป็น Query or Reply
flags

flags -> query or reply, recursion desired, recursion available, reply is authoritative?

Content

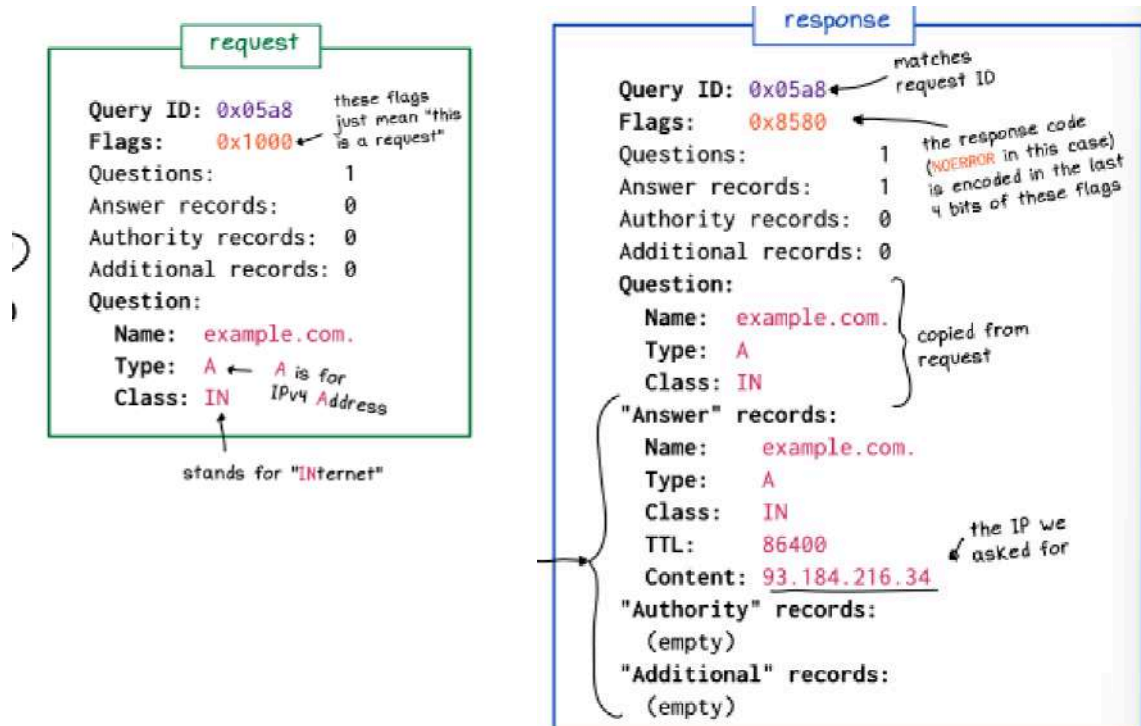
question -> query name

answers -> RRs ที่ตอบ Query มา

authority -> server resources

additional info -> optional more info

DNS PACKET



How to จดทะเบียน DNS

1. เข้า registrar เช่น networkutopia.com
2. ค่าจะเพิ่มโดเมนเว็บเราไว้ใน TLD server database with Authoritative Name Server ของเรา
3. เราก็เพิ่มAuthoritative server ได้เลย

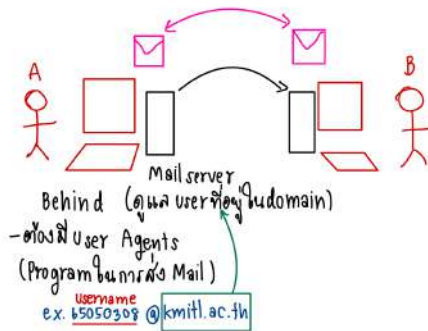
DNS usually got attacked by DDoS and Spoofing

DDOS -> not quite successful while bombarding root servers but successful on TLD servers

Spoofing -> มีพวกดักจับและส่งการตอบกลับปลอมๆให้

Week 6: Application layer Ep.3 : Mail(SMTP), Streaming, CDNs

- Protocol ที่ใช้ส่ง mail : SMTP เป็น TCP port 25, IMAP, POP3



ถ้าอยากดูว่า domain name ของสจล. Gateway คืออะไร :
nslookup -type=MX kmitl.ac.th

คำตอบที่ได้คือ ชื่อ mail gateway ของ สจล

```
(base) nattawadeekwankao@Nattawadees-MacBook-Air ~ % nslookup -type=MX kmitl.ac.th
Server:      192.168.0.1
Address:     192.168.0.1#53

Non-authoritative answer:
kmitl.ac.th mail exchanger = 10 alt4.aspmx.l.google.com.
kmitl.ac.th mail exchanger = 5 alt1.aspmx.l.google.com.
kmitl.ac.th mail exchanger = 1 aspmx.l.google.com.
kmitl.ac.th mail exchanger = 11 mailsecure1.kmitl.ac.th.
kmitl.ac.th mail exchanger = 10 alt3.aspmx.l.google.com.
kmitl.ac.th mail exchanger = 5 alt2.aspmx.l.google.com.

Authoritative answers can be found from:
```

Mail server : aspmx.l.google.com.

- Mail Component

หน้าที่ของ Mail server => คือ ไปรษณีย์ของ user ที่อยู่ในสังกัดนั้น และในไปรษณีย์ก็จะมี mail server ของเรา ถ้าแกะออกมาจะมี 2 ส่วนคือ

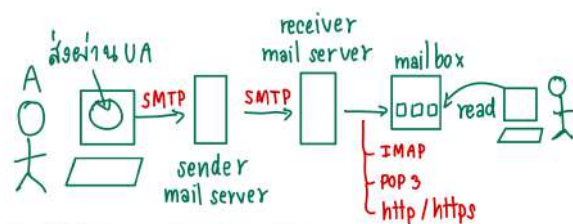
1. User mailbox (ซึ่งจะแตกต่างกันไปตามแต่ละบุคคล) เช่น 65050308
2. Outgoing Msg queue ถ้ามี user คนไหนส่งเมลมาก็จะเอาใส่ไป queue และ feed ออกมาเมื่อถึงคิว ทำงานในกรณีที่เรเป็นผู้ส่ง

กรณีที่เป็นคนส่ง : จะทำการดูว่า mail server จำหน่ายของไปถึงใคร และ forward ไปยัง mail gateway ของคนนั้น

กรณีเป็นคนรับ : พอรับจะดูว่า mail นี้จำหน่ายของถึง user คนไหนของเรา แล้วค่อยเอาจดหมายนี้ยัดลงไป mailbox ของคนนั้น

User agents => เป็นโปรแกรมที่ใช้ในการส่ง mail ในการเปิดเมลก็จะมีโปรแกรมของตัวเอง เช่นในบางองค์กรจะใช้ Ms outlook ซึ่งถือว่าเป็น user agents แต่ปัจจุบันนิยมส่งผ่าน browser

- Mail Protocols : เป็นส่วนที่ mail server ใช้ในการคุยกัน



Hop ① : UA → Sender mail server

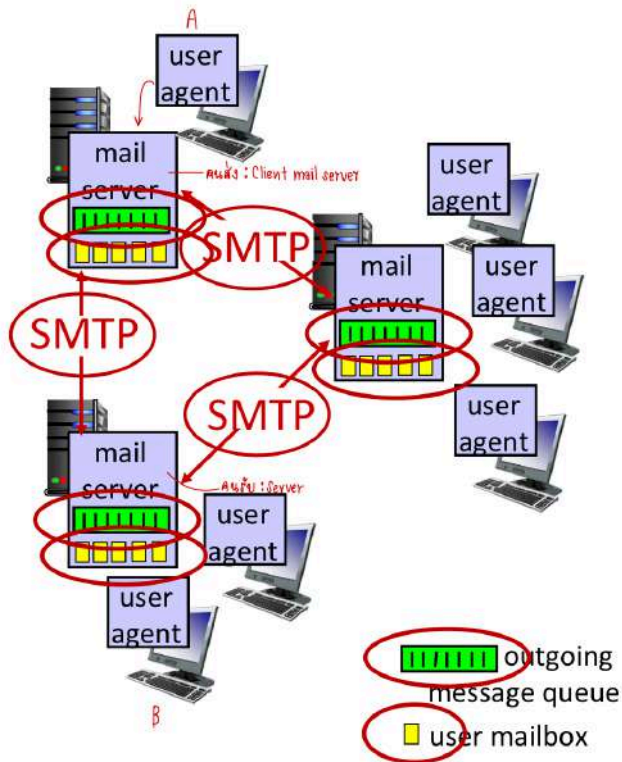
Hop ② : Sender mail server → receiver mail server

Hop ③ : receiver mail server → mail box (ขึ้นกับ user ว่าเปิดเมลด้วยอะไร)

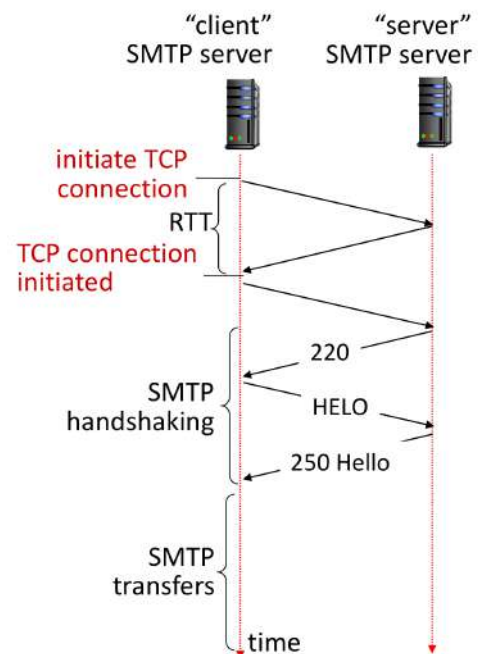
Component ที่เกี่ยวข้องในการทำ email

- User agents : เป็นตัวอ่านหรือส่งเมล eg. outlook, iphone mail client
- Mail server สังกัด email ของผู้ส่งและผู้รับ มี mailbox และ message queue

- SMTP (Simple mail transfer protocol) ใช้ในการคุยกันระหว่าง mail server แต่ละตัว รวมถึง ua เวลาคุยก็ใช้ smtp



SMTP port 25 ซึ่ง mail ก็เหมือน web จะต้องได้รับครบทุกส่วน จึงใช้ TCP ที่มีการ initiate connection หรือ 3-ways handshake หรือ 3 message ดังนั้นในการสื่อสารจริงๆจะเริ่มที่ รอบที่ 4



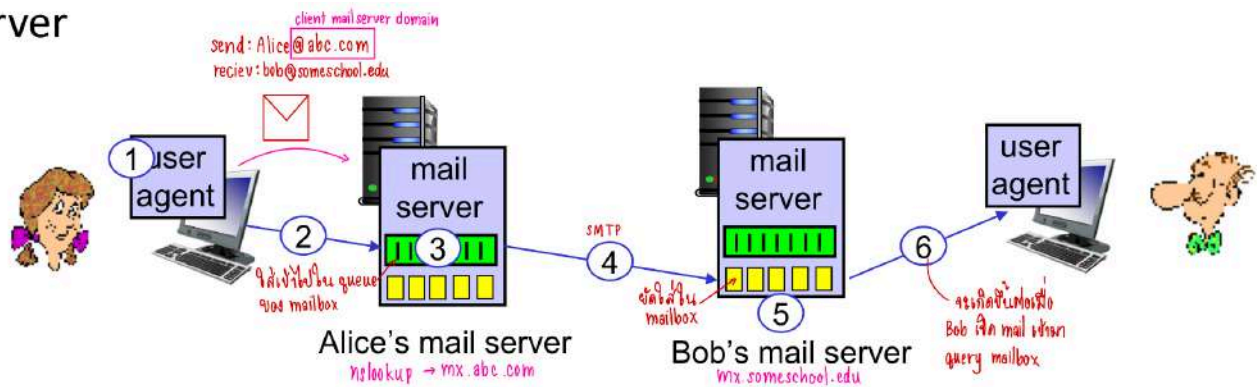
SMTP จะแบ่งการทำงานออกเป็น 3 Phase

1. SMTP handshaking (greeting) เป็นการคุยกันระหว่าง Client mail server กับ Server หลังจากการทำงานของ TCP server จะเริ่มคุยกับ Client ก่อนโดยการส่ง message "220" มาบอก Client มาเพื่อบอกว่าพร้อมรับข้อมูลแล้ว จากนั้น Client ก็จะส่งข้อมูลให้ แล้ว server ก็จะมีการส่งกลับมามาว่าได้รับแล้ว
2. SMTP transfer of messages หลังจาก handshaking เรียบร้อยแล้ว Client ก็จะเริ่มส่งข้อมูล
3. SMTP closure เมื่อส่งข้อมูลครบแล้ว ก็จะจบการทำงาน

command/response interaction (like HTTP)

- **commands:** ASCII text
- **response:** status code and phrase

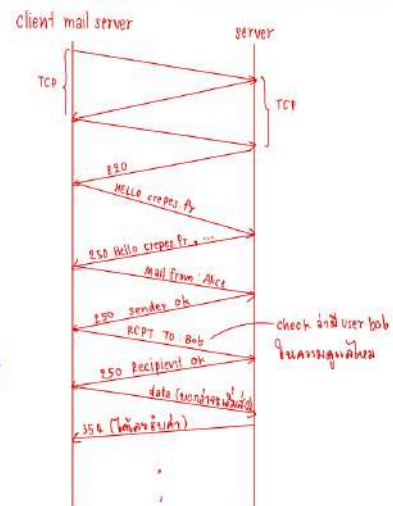
rver



Sample SMTP interaction

sender receiver
alice@crepes.fr => bob@hamburger.edu

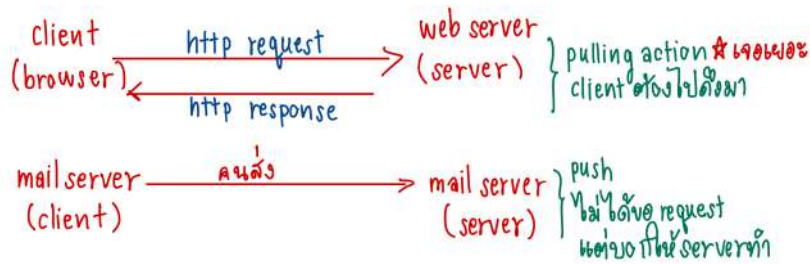
S: 220 hamburger.edu
C: HELLO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM : <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO : <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: Data
S: 354 enter email, end with "." On a line by itself
C: how do you like ketchup ?
C: how about pickle ?
C: .
S: 250 message accepted for delivery
C: quit => บอกว่าไปแล้ว
S: 221 hamburger.edu closing connection



Application

แต่กรณีของ Mail กับ Web ไม่เหมือนกัน

- Web มี client เป็น browser และมี server ที่เวลาคุณต้องการส่ง request ไปก่อนเสมอ
- HTTP : Client pull ถ้า text มีทั้ง รูปเสียง ภาพ ก็จะเป็นคนละ object ดังนั้น http request ก็จะเป็นคนละตัว ใช้ 8 bit ASCII จบด้วย /r/n
- SMTP : Client push ถ้า text มีทั้ง รูปเสียง ภาพ จะส่งไปเป็น mail เดียวแต่มีการแบ่ง message ข้างใน และใช้ persistent connection ใช้ 7 bit ASCII จบด้วย .
- persistent connection เป็นรูปแบบ connection ที่ให้เราสามารถส่งได้หลาย HTTP Request ภายใน TCP connection



Mail message format

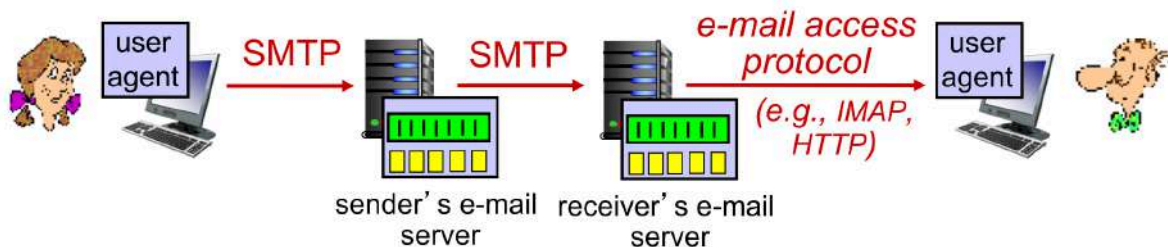
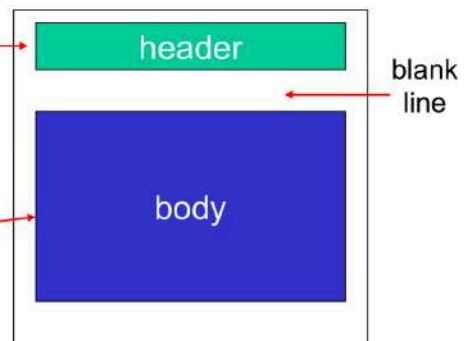
- SMTP มี RFC 5321
- RFC 2822 เป็น syntax ของ mail message ว่าถ้าอ่าน email จะมี tag คล้ายๆ html

header lines, e.g.,

- To:
- From:
- Subject:

these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!

Body: the "message", ASCII characters only



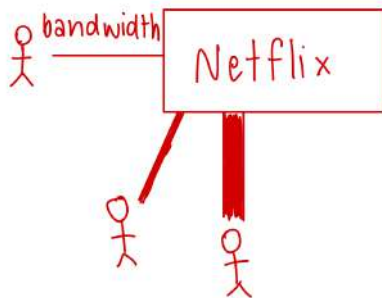
การดึงข้อมูลจาก user agent ของคนรับ ดึงข้อมูลมาอ่าน ไม่ได้ใช้ SMTP แต่จะใช้ eg. IMAP, POP3, HTTP, HTTPS (ปัจจุบัน)

POP (Post office protocol) : จะเปรียบเทียบ mail server เหมือนที่ทำการไปรษณีย์ และ mailbox คือตู้ไปรษณีย์. ของเรา ดังนั้นคนที่มาเปิดจดหมายก็ต้องมาดึงจดหมายไปก็คือ Post office

ถ้าเป็นสมัยแรก คอยกันด้วย POP3 พอเข้าไปอ่าน mailbox มันจะเอาจดหมายนี้ออก ไปแล้วไปเก็บไว้ที่ program ที่เปิด POP3 ประมาณ 20 ปี ถึงจะมาอ่านและส่ง mail ในรูปแบบของ web => http แทน

Video Streaming and CDNs : Context

- ปริมาณการใช้ internet บน Application เป็นประเภท video streaming เยอะมาก ซึ่งผลสำรวจจาก traffic ของคนที่ใช้ internet ซึ่งส่วนใหญ่มาจากพวก video streaming ทั้งหมด
- Challenge : มีลูกค้าหรือคนเข้าถึง streaming จำนวนมาก และ user แต่ละคนก็มีความสามารถไม่เท่ากัน บางคนเข้า net ที่ bandwidth สูงต่ำไม่เท่ากัน



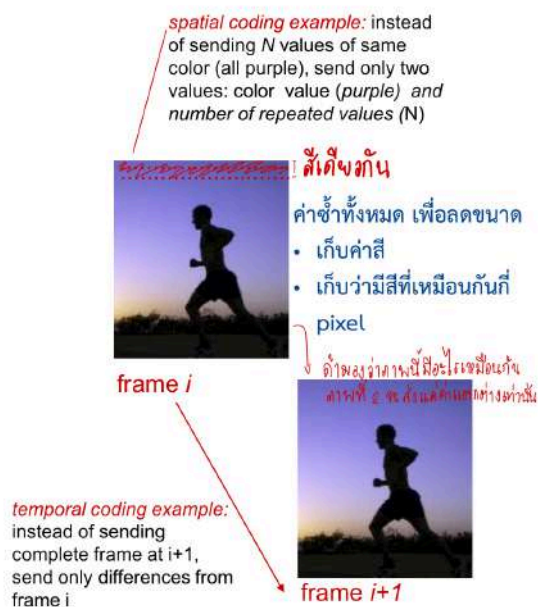
Netflix ต้องคิดว่าเวลาที่ปล่อย content ออกมาต้องปล่อยแบบไหนถึงจะได้ประสิทธิภาพที่สูงที่สุด

ถ้ามี server ที่เดียวแต่มี user ทั่วโลก จะมีปัญหาเกี่ยวกับระยะทาง

ณ ปัจจุบัน ค่าใช้ network ที่มีอยู่ โดย application-level infrastructure

Multimedia : Video

- จะเก็บข้อมูลออกเป็น sequence ของภาพ ดังนั้นในภาพเคลื่อนไหวที่เราเห็น อาจจะประกอบไปด้วยภาพนิ่งที่ต่อเนื่องกัน แล้วเอามาต่อกันอย่างรวดเร็ว
- แต่ละภาพนิ่งเก็บข้อมูลเป็น array of pixels ซึ่งภายใน pixel จะเก็บค่าสี ขาวดำค่าสีใช้แค่ 1 byte หรือ 8 bit ในการเก็บค่าสี 0(white)-255(black) ในการเก็บข้อมูลของภาพ กว้างxยาว = ... byte
- ถ้าเป็นภาพสี RGB แต่ละ pixel เก็บ 3 byte จึงถือว่ามีขนาดค่อนข้างใหญ่ เมื่อมองในมุมว่ามันเป็น video ทำให้ต้องมีพื้นที่การเก็บที่มาก
- เวลาส่งผ่าน bandwidth ก็ต้องส่งทั้งหมด แต่ขึ้นอยู่กับความสามารถของ bandwidth ที่รับส่งได้ จึงต้องทำให้มันลดลง

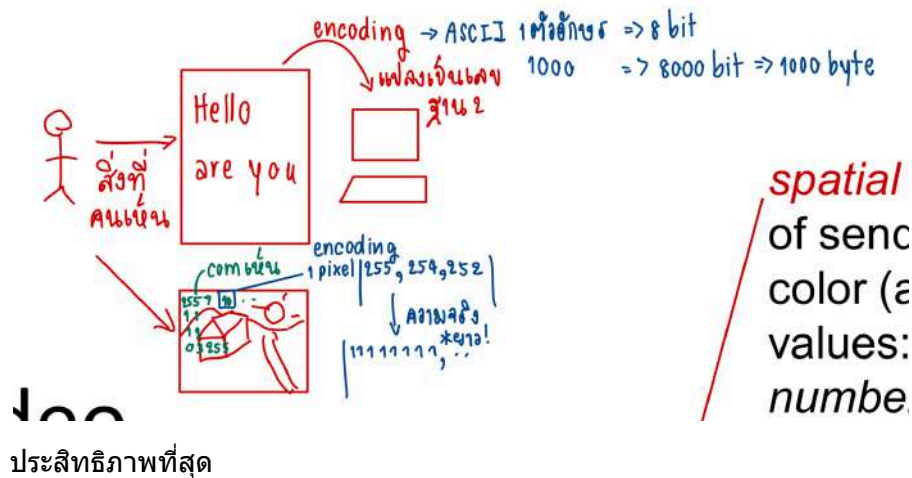


Spatial (within image) : วิธีที่ใช้ลดจำนวนการเก็บของข้อมูล ถ้า pixel ค่าเดียวกัน จะเก็บแค่ค่าสี และจำนวน pixel ที่เหมือนกัน

Temporal (from one image to next) : เก็บเฉพาะความแตกต่างของ 2 ภาพที่ติดต่อกันว่า pixel นี้ไม่เหมือนกัน

เป็นการช่วยลดข้อมูลที่ต้องส่ง

ในกรณีถ้าข้อมูลเป็น text



ถ้ามีภาพเพิ่มก็จะดูว่าแตกต่างจากภาพข้างบนมากแค่ไหน และต้องหาวิธีเพื่อลดขนาดของข้อมูล ดังนั้น encoding คือการแปลงสิ่งที่คนเห็นให้กลายเป็นสิ่งที่คอมพิวเตอร์เข้าใจ ต้องหาวิธีที่แปลงแล้วได้ขนาดหรือเก็บ storage น้อยที่สุด ถ้ามีการส่งข้อมูลนี้ผ่าน internet จะทำให้ใช้ bandwidth ได้มี

ประสิทธิภาพที่สุด

- CBR (Constant bit rate) : ในภาพเคลื่อนไหว ประกอบด้วยหลายภาพต่อกัน ดังนั้นเวลาเก็บข้อมูล จึงจะต้องมีการหั่นเป็นส่วนๆ หรือ fixed rate ในการ encode
- VBR (Variable bit rate) : ไม่ได้หั่นด้วย rate ที่เท่ากัน

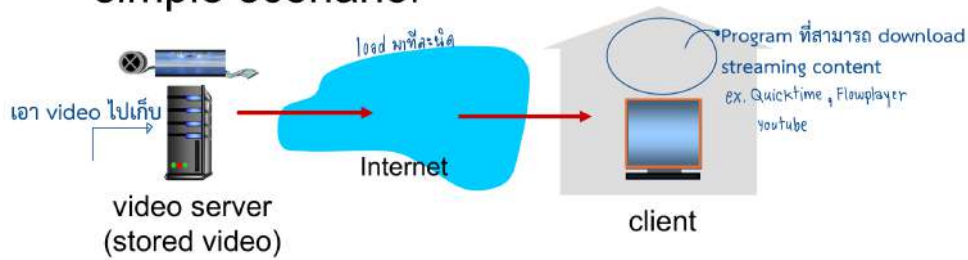
examples:

- MPEG 1 (CD-ROM) 1.5 Mbps (เก็บได้น้อย encoding rate)
- MPEG2 (DVD) 3-6 Mbps (เน้นสูงกว่า)
- MPEG4 (often used in Internet, 64Kbps – 12 Mbps)

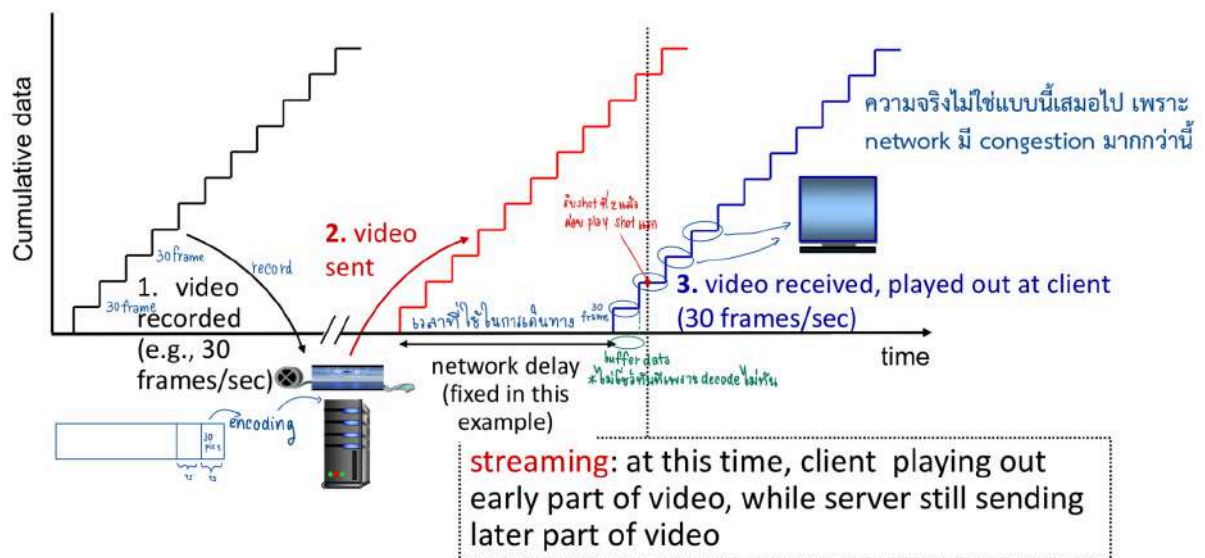
Streaming stored video

- ในสมัยก่อน youtube ไม่สามารถ save ได้ และมีเอาไว้ให้บริษัทเพลงแนะนำให้คนรู้จัก แต่ไม่ให้ copy ไปฟัง ห้าม download แต่ให้ไปซื้อได้ แต่มีคนไปเขียนโปรแกรมให้ดู content ของ youtube ลง file ดังนั้น youtube จึงเปลี่ยน business model
- เมื่อก่อนเวลาดู youtube จะต้องกดหยุดเพื่อรอให้ download เสร็จก่อนค่อยดู เพื่อป้องกันการกระตุก เพราะถ้ามันโหลดไปก่อนแล้ว มันจะอยู่บน cache บนเครื่องเราเรียบร้อยแล้ว ดังนั้นมันจะไม่กระตุกหรือค้าง
- แต่ในปัจจุบันไม่ต้องรอนาน เพราะมันจะทยอย download และเก็บลง buffer เรื่อยๆ ทำให้ไม่มีความรู้สึกว่ามันช้า
- Youtube เป็น Client ที่มีความฉลาด คือมันจะมีการประเมินสถานการณ์ของ internet ว่าจะใช้ความเร็วเท่าไร

simple scenario:

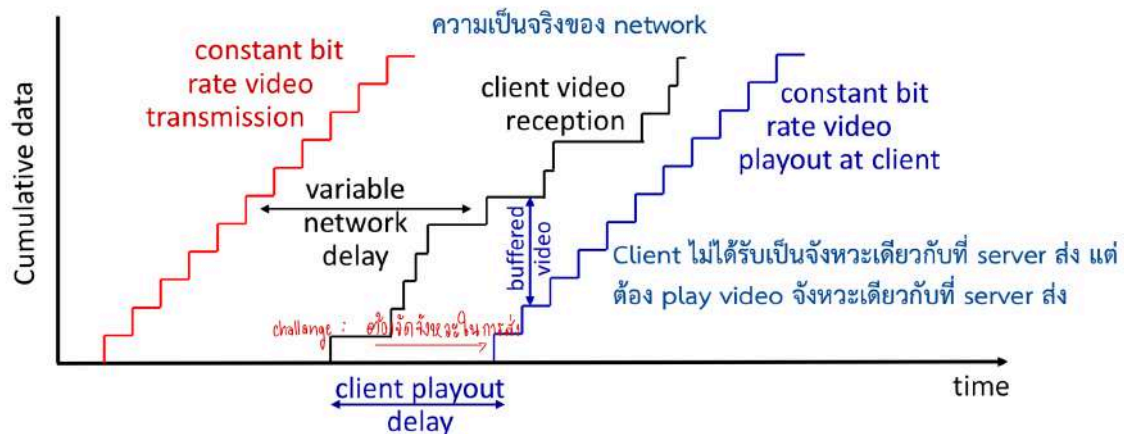


เวลาส่ง video streaming เก็บอย่างไร



Playout timing : เป็น sampling ฟังคนส่ง

ด้วยความที่ network delay จึงต้องมี client-side buffer เพื่อที่จะทำให้ดึงจาก buffer ไปใช้ได้ไม่สะดุด แต่ client สามารถหยุด ไปข้างหน้า หรือย้อนกลับได้ ดังนั้นคนทำ application video streaming ฟัง client ไม่ได้แค่ download buffer play แต่ต้องมี command ในการส่งระหว่าง client กับ server เช่น pause, fast-forward, rewind, jump through



ในการจัดการ buffer ไม่ได้มีแค่ช่วงแรกช่วงเดียว แต่ buffer ฝั่ง client เป็นเหมือนตัวเก็บน้ำที่จะปล่อยน้ำออกในปริมาณที่คงที่เพื่อให้เหลือในการปล่อยครั้งถัดไป

Streaming multimedia : DASH (Dynamic, Adaptive Streaming over HTTP)

- Youtube ใช้ protocol http ดังนั้นตัว player บน youtube เวลาคุยกับ server เลยสร้าง Protocol DASH ขึ้นมา เพื่อใช้จัดการ content ที่ play ได้ง่ายขึ้น

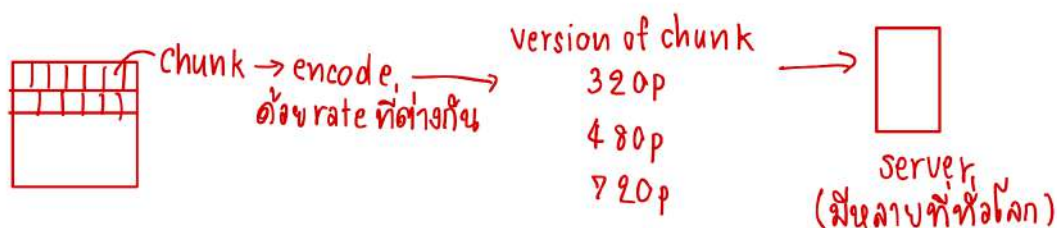
- Server

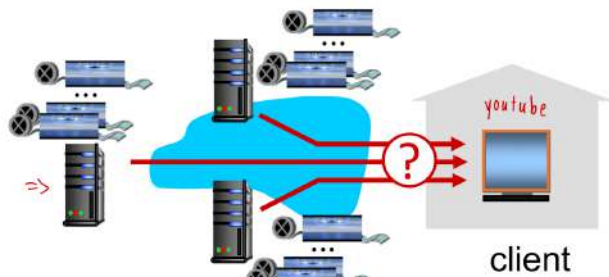
จะทราบว่า client มีความสามารถแตกต่างกัน บางทีอาจจะเจอ client ที่มี bandwidth ต่ำ ดูได้แค่ 480p บางคนอาจดูได้ 720p หรือมากกว่านี้ ดังนั้นวิธีในการเก็บไฟล์จึงต้องมีการแบ่งภาพออกมาเก็บเป็นช่วงๆ เรียกว่า chunk ซึ่งจะแบ่งตามเวลา เช่นกำหนดเวลา 1 วิ 30 frame ก็จะตัดทุก 1 วิ ซึ่งแต่ละ chunk อาจจะข้อมูลไม่เท่ากัน

Manifest file เป็นตัวเก็บว่า ~~chunk~~ ^{node} มี node ไหนบ้างที่เก็บ chunk นี้

- Client

ดู server-to-client bandwidth ว่าตัวไหนเป็นอย่างไร และดูที่ Manifest file ว่าควรดาวน์โหลดจากเครื่อง server ไหน





พอเข้ามาจะไปตาม server => จะดูว่า client bandwidth อันไหนส่งได้มากที่สุด ip address geolocation และจะเลือกตัวที่ใกล้สุดมาให้ =>

การทำงานของ DASH

- Player สามารถเลือกความละเอียดของข้อมูลได้ ดังนั้น มุม Client ไม่ได้คิดว่าข้อมูลที่ได้ขาดหาย แต่เพียงขัต้น้อยลงแต่สามารถเห็นข้อมูลต่อเนื่องได้ และมันจะรู้ว่าเมื่อไหร่ควร request chunk ใหม่
- ควรเลือก chunk ไหนในการ request และ encoding base ตัวไหน

Streaming video = encoding + DASH + playout buffering

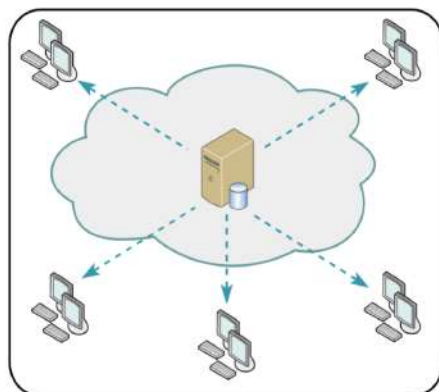
Trai

Buffer ต้องรู้ตัวเองว่าต้อง buffer ข้อมูลขนาดไหน, สามารถ adjust ขนาด buffer ได้ และ protocol ที่ใช้ พอรู้ว่าต้องปรับความคมชัด จะทำการ request chunk ใหม่

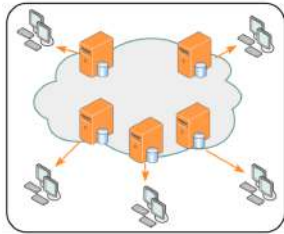
Content distribution networks (CDNs) : network ที่มี server กระจายทั่วโลก ตาม ISP แล้วค่อยไปซื้อ แล้วเอา content ไปวางตาม server

Challenge: how to stream content (selected from millions of videos) to hundreds of thousands of simultaneous users?

- Option 1 : คนที่จะเข้ามาติดต่อกับเราต้องผ่าน server ใหญ่ mega-server ตัวเดียวที่เก็บ content streaming ทั้งหมด แต่ถ้าตัวนี้เสียหายก็พังหมด ,ทุกคนต้องมาถาถามัน (point of congestion) และระยะทางที่ไกล ซึ่งไม่เหมาะกับ user ในระบบเยอะ เข้าถึงได้อย่างรวดเร็ว และข้อมูลเยอะบางทีการมี server เดียวอาจจะไม่ตอบโจทย์



- Option 2 : ถ้าในกรณีที่มีหลาย server ที่กระจายวางไว้ทั่วโลก เช่น netflix ที่ไม่ได้เป็นเจ้าของ server แต่ซื้อ service และบอกคนที่ดูแล server นี้ว่าช่วยเอาหนังไปวางที่ area ต่างๆ ทั่วโลก และทำการจ่ายเงิน

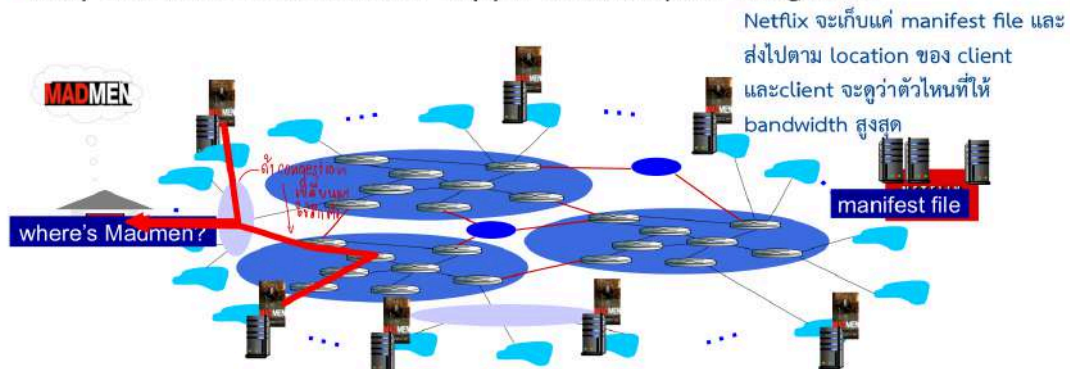


Akamai : เป็น CDNs Provider มี content กระจายทั่วโลก

360K servers	100+ million hits per second	7+ trillion deliveries per day	175+ terabits per second (250+ peak)
4,200+ locations	1,350+ networks	840+ cities	135 countries

How does Netflix work?

- Netflix ไม่ได้เป็นเจ้าของ CDNs แต่ไปซื้อ service ซึ่งจะดูแลแค่ server หลักที่เป็น core ในการเก็บ content หลัก
 - Netflix: stores copies of content (e.g., MADMEN) at its (worldwide) OpenConnect CDN nodes
 - Copy เก็บไว้ใน CDNs หลายที่ ตามความต้องการของลูกค้า ไม่ต้องวางทุกที่
 - subscriber requests content, service provider returns manifest
 - using manifest, client retrieves content at highest supportable rate
 - may choose different rate or copy if network path congested



Transport Lay

Week 7: Transport Layer Protocol Ep.1 : Socket, UDP, Reliable Transfer Protocol Concept.

Transport layer VS Network layer

- Transport layer สื่อสารระหว่าง Processes
- Network layer สื่อสารระหว่าง Hosts

ชั้น Transport Layer ทำหน้าที่ใส่ Segment Header ให้กับข้อมูลที่ต้องการส่ง

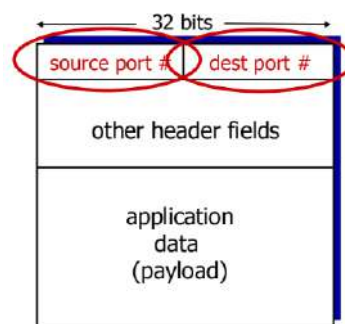
*Segment Header คือส่วนที่ทำหน้าที่เก็บข้อมูลที่จำเป็นในการควบคุม/จัดการการสื่อสาร คล้ายเจ้าหน้าที่จราจร

Transport layer มีโปรโตคอลหลัก 2 แบบคือ UDP และ TCP

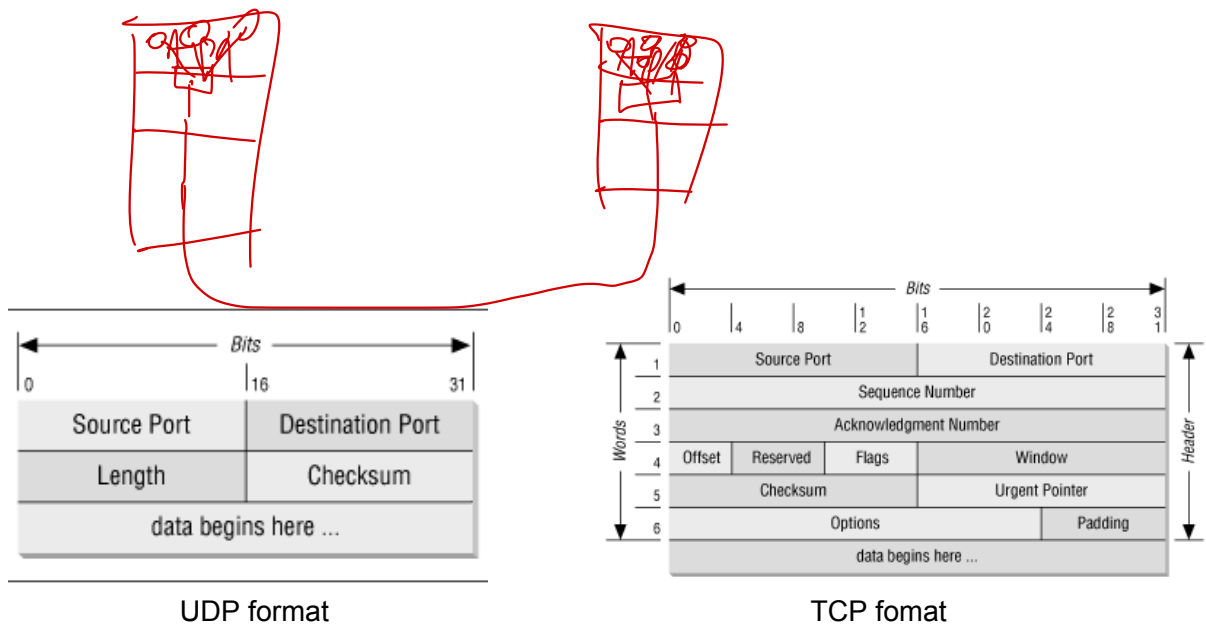
TCP	UDP
Secure	Unsecure
Connection-Oriented	Connectionless
Slow	Fast
Guaranteed Transmission	No Guarantee
Used by Critical Applications	Used by Real-Time Applications
Packet Reorder Mechanism	No Reorder Mechanism
Flow Control	No Flow Control
Advanced Error Checking	Basic Error Checking (Checksum)
20 Bytes Header	8 Bytes Header
Acknowledgement Mechanism	No Acknowledgement
Three-Way Handshake	No Handshake Mechanism
DNS, HTTPS, FTP, SMTP etc.	DNS, DHCP, TFTP, SNMP etc.

- Multiplexing คือกระบวนการรวมข้อมูลจากหลายแอปพลิเคชันบนเครื่องเดียวกันแล้วส่งผ่านช่องทางเดียวไปยังปลายทาง
- Demultiplexing คือกระบวนการแยกข้อมูลที่ได้รับการตรวจสอบหมายเลขพอร์ตต้นทางและปลายทางใน segment header และส่งต่อไปยังแอปพลิเคชันที่ถูกต้องบนเครื่องปลายทาง

This is Datagram



TCP/UDP segment format



Multiplexing, demultiplexing: based on segment, datagram header field values

UDP: demultiplexing using dest IP, dest port แต่ถ้าถึงปลายทางแล้วใช้ dest port อย่างเดียว

TCP: demultiplexing using 4-tuple:

- source IP address
- source port number
- dest IP address
- dest port number

Multiplexing/demultiplexing เกิดขึ้นทุกเลเยอร์

ใน UDP เรียก packet ในระดับของตัวเองว่า Datagram ส่วน TCP เรียกว่า Segment

- แต่หากเรียกทั่ว ๆ ไปไม่ได้เจาะจงก็ใช้ packet เฉย ๆ

Knowledge Check

- If the UDP application at the server has 10 Client Connections.
 - How many UDP Sockets ? $\rightarrow 1+10 = 11$
 - Clarify how many is/are UDP server socket (s), how many is/are UDP client socket(s)
- If the TCP application at the server has 10 Client Connections.
 - How many TCP Sockets ? $\rightarrow 10+10 = 20$
 - Clarify how many is/are TCP server socket (s), how many is/are TCP client socket(s)
- True or False
 - All UDP Client socket's port in UDP application in Q.1 must use the same port number. **F**
 - All UDP Server socket's port in UDP application in Q.1 must use the same port number. **T**
 - All TCP Client socket's port in TCP application in Q.2 must use the same port number. **F**
 - All TCP Server socket's port in TCP application in Q.2 must use the same port number. **T**

→ มัน fork process port เลขเดิม 30

UDP: User Datagram Protocol

ถ้าจะดูว่าคอมเรามี socket อะไรบ้างเกิดขึ้นอยู่ TCP or UDP ก็ connection ให้ใช้คำสั่ง " netstat "

- เป็น best effort service: ส่งไปแล้ว รับได้แค่ไหนก็แค่นั้น แสดงว่า lost, ไม่มีผู้รับ, ไม่เรียงลำดับได้

- เป็น connectionless: ไม่มีการ handshaking ระหว่าง UDP sender, receiver เลย
- Ex. app ที่นำ UDP ไปใช้
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS => ในหนึ่งวัน query เป็นล้าน
 - SNMP (Simple Network Management Protocol) => ใช้ใน nw management
 - HTTP/3
- ส่วน UDP ที่อยู่ใน HTTP/3 => QUIC จะมีการเพิ่ม reliability, congestion control เข้ามาใน application layer

UDP: Transport Layer Actions

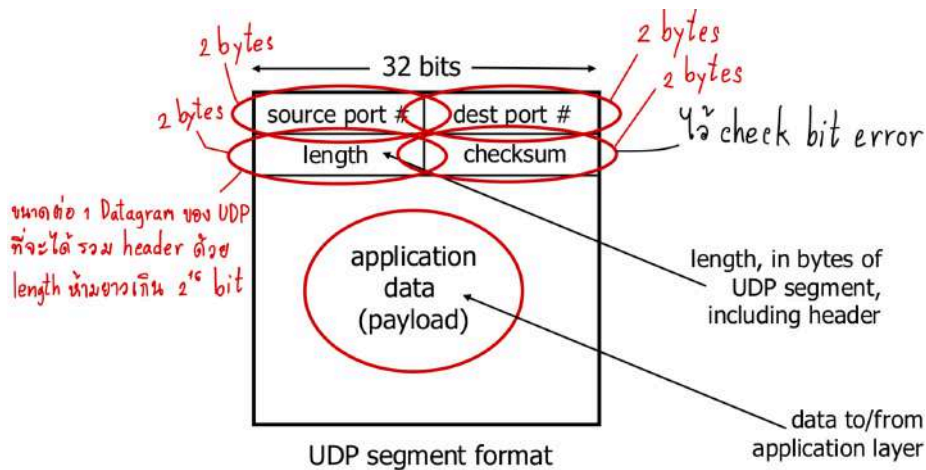
ฝั่ง sender:

- ส่ง application-layer message
- กำหนดค่าของ UDP segment header field
- สร้าง UDP segment
- ส่งต่อไปชั้น IP (network layer)

ฝั่ง receiver:

- รับ segment จาก IP
- ตรวจสอบค่าของ field checksum ใน UDP header
- สกัด application-layer message ออกมา
- demultiplex message ขึ้นไป application ผ่าน socket

UDP == RFC 768



Summary: UDP

- segment อาจจะ lost ได้ ส่งแบบ out of order ได้ (ไม่เรียงลำดับ)
- ส่งไป หวังว่าจะรับได้

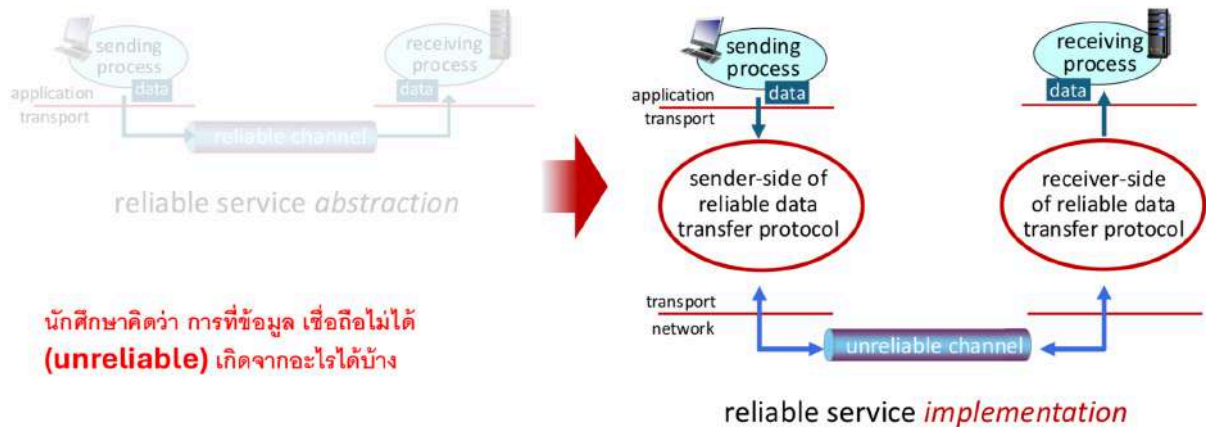
ข้อดี UDP เพิ่มเติม

- ไม่ต้อง setup/handshaking
- เครื่อง client ล่มก็ยังสามารถส่งได้

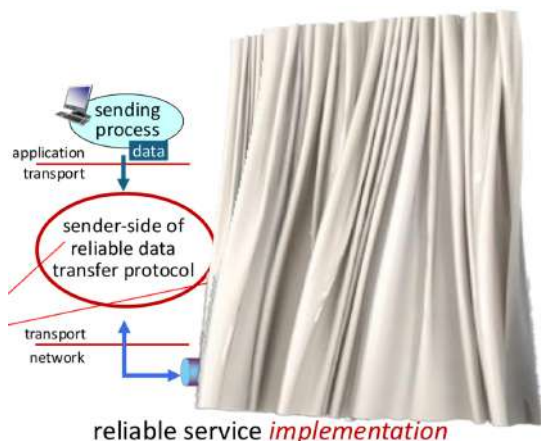
Principles of reliable data transfer

ด้วยความที่ internet ไม่มีความน่าเชื่อถือใด ๆ ทั้งสิ้น ไม่มี security, reliable service

ดังนั้นจึงต้องทำ reliable server ขึ้นมา ซึ่งมีกระบวนการบางอย่างก่อนถึงจะส่งไปยัง unreliable channel



โดยที่ความ complexity ของ reliable data transfer protocol ก็จะขึ้นอยู่กับลักษณะของ unreliable channel เช่น lose, corrupt, reorder data



ด้วยความที่ sender และ receiver ไม่รู้สถานะของกันและกัน เช่น message ที่ส่งไปนั้น receiver ได้รับไหม
<<<

How to make it reliable?

1.0 Bit error = ถ้าหาก data ที่ส่งไปเกิด flip bits แล้ว receiver จะรู้ได้ยังไงว่ามันผิด

>> ใช้ checksum เพื่อ detect bit errors

2.0 แล้วมนุษย์ recover errors ระหว่าง conversation ได้ยังไง?

>> Receiver ต้องตอบว่าได้รับ data ถูกต้อง หรือไม่

- Acknowledgements (ACKs): receiver ตอบกลับว่าข้อมูลถูกต้อง
- negative acknowledgements (NAKs): receiver ตอบกลับว่า packet (pkt) มี error
 - Sender retransmit (ส่งใหม่) pkt เมื่อเจอ NAKs
- Stop and wait: sender ส่งไป 1 packet แล้วรอ receiver ตอบกลับ

- ปัญหา: ถ้า ACK/NAK corrupt แปลไม่ออกว่าเป็นอะไร จะ retransmit อย่างเดียวไม่ได้ => เพราะ pkt จะมันซ้ำ คิดในกรณี worst case คือ จริงๆ ได้รับแต่ว่าดันตอบ NAK เลยเกิด duplicate pkt

2.1

Week 8: Transport Layer Protocol Ep.2 : TCP

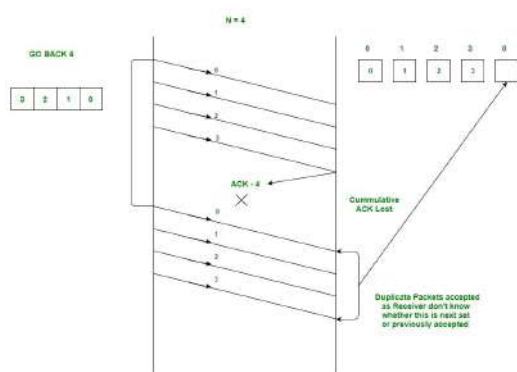
- TCP overview

TCP : Point-to-point protocol เครื่องหนึ่งส่งให้อีกเครื่องหนึ่ง ที่สามารถเชื่อถือได้

in-order byte stream :

full duplex : จริงๆ UDP ก็เป็น แต่เรียนแค่ข้างเดียวเลยเป็น half duplex และเวลาส่งจริงเป็น bi-directional และมี MSS (Maximum segment size) ทำให้เวลาส่งจึงต้องแบ่งเป็น TCP หลายๆ segment

Cumulative ACK :



จะไม่ส่งตัวที่เหลืกลับมา เพราะถือว่า 1,2,3 เกิดขึ้นไม่ได้ ถ้าไม่ได้รับ message 0

Pipelining : ส่งเป็น window slice

Connection-oriented : มีการมองตา connect กันก่อนถึงเริ่มส่ง เรียกว่า handshaking

Flow controlled : มีการบอก sender ว่ารับได้แค่นี้ ห้ามส่งมากเกินไปมันจะ overflow

- TCP segment structure

acknowledgement number : เกี่ยวกับฝั่งตรงข้าม ที่บอกเราว่าเขาได้รับข้อมูลจากเรามาแล้วไหน

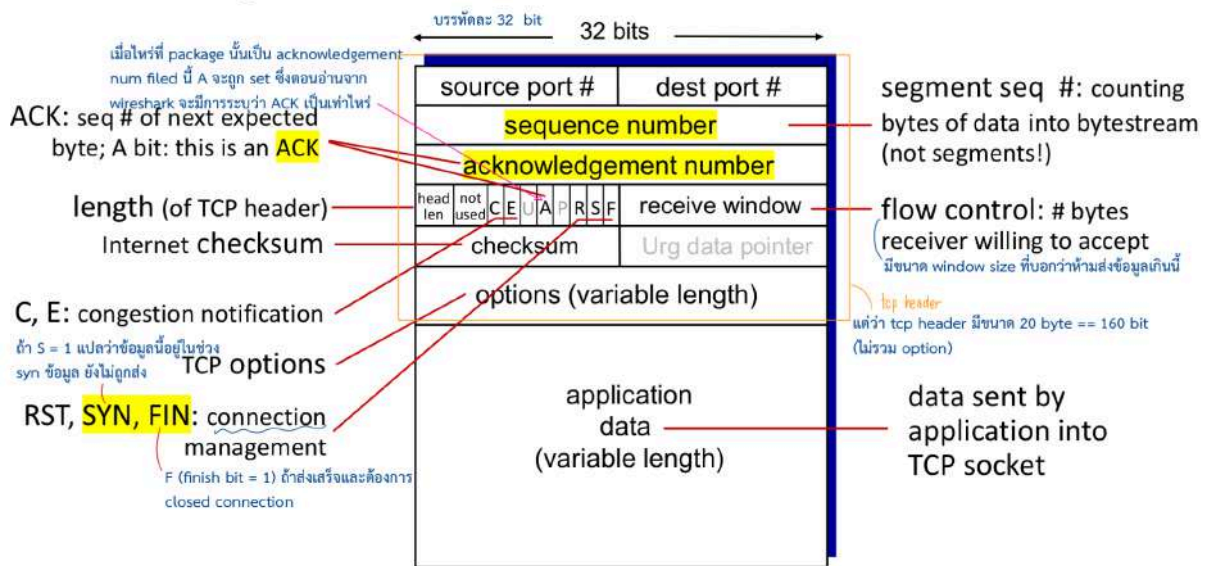
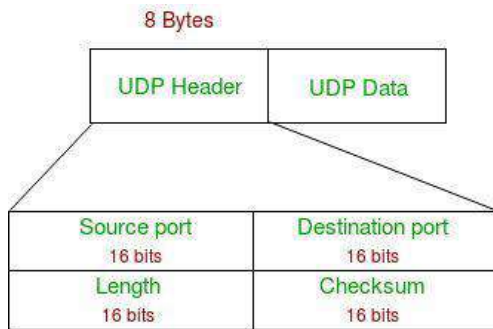
checksum :

Checksum คือการตรวจสอบความถูกต้องของข้อมูล โดยฝ่ายที่ส่งข้อมูลจะทำการคำนวณค่า checksum แล้วใส่ค่า checksum ลงไปใน header ของแต่ละ Protocol ข้อมูล เช่น TCP/IP ,UDP,ICMP และเมื่อฝ่ายรับข้อมูล รับข้อมูลมาแล้วจะนำเอาค่า checksum นี้มาตรวจสอบข้อมูลที่รับมาว่าถูกต้องหรือไม่ เพราะการตรวจสอบความถูกต้อง และสมบูรณ์ของไฟล์โปรแกรมที่ดาวน์โหลดจากเว็บไซต์ก่อนการติดตั้งนั้นเป็น สิ่งสำคัญ เพราะไม่เช่นนั้นท่านจะเสี่ยงต่อความเสียหายที่จะเกิดขึ้นกับโปรแกรมมาโคร จัน หรือ โปรแกรม Back door ที่แทรกมากับไฟล์ดังกล่าว

ตัวอย่างโปรแกรมที่ใช้ในการ CheckSum เช่น md5 CheckSum, winMD5sum, hash เป็นต้น

มีเหมือนกับ UDP

UDP มี header 4 อัน ได้แก่ (1byte == 8bit)



Transport Layer: 3-6

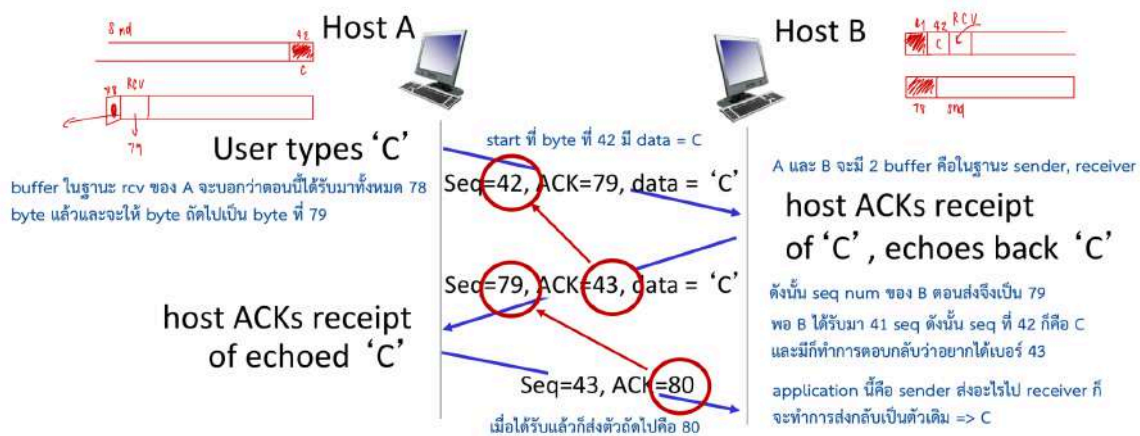
TCP header มีขนาดที่เยอะแต่ก็แลกมาด้วยการส่งข้อมูลที่ครบถ้วนและเรียงตามลำดับ
example : HTTP, SMTP

- TCP คิด Sequence number และ ACK ยังไง

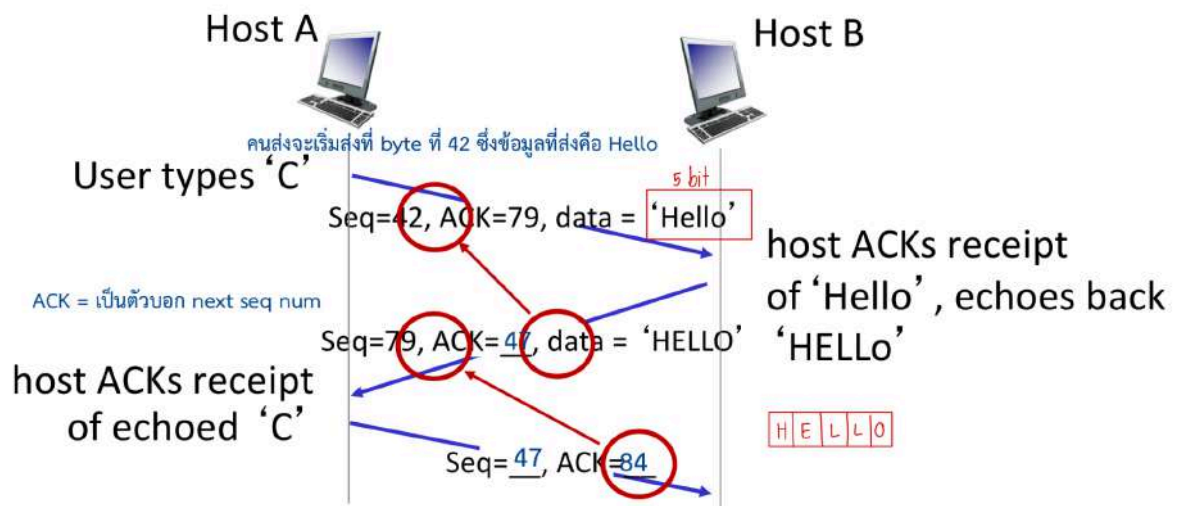
Sequence number : มีขนาด 32 bit สามารถเป็นค่าได้ระหว่าง 0 ถึง $2^{32} - 1$ TCP นับ sequence number เป็น byte เลยเรียกว่า byte stream ดังนั้นจะมองว่าในการส่งต่อ segment = 1 stream
eg. ถ้า 1 stream มี 5 byte ก็จะทำให้การส่ง 5 byte/stream ถ้า sequence num start = 1 ก็จะเริ่มที่ 1

ปกติ sequence number ไม่ได้ถูกกำหนดว่าจะต้องเริ่มที่ 0 เสมอไป แต่ปกติจะเริ่มที่ 0

Acknowledgement : เมื่อ A ส่งข้อความหา B จะมีการบอกว่า A ส่งจาก sequence number start ที่ ... และ B จะ ACK ตอบกลับมามากกว่า ได้รับถึงเท่าไร
แต่ TCP มีความ special ถ้าเราส่ง sequence number เริ่มต้นที่ 10 package มี 1 byte TCP จะส่ง ACK กลับมาเป็น 11 มันคิดว่า seq num ถัดไปที่ตัวเองจะได้รับต้องนำหน้าด้วย 11 เพื่อเป็นการบอกฝั่งส่งว่ารอบถัดไปต้องเริ่มต้นด้วย seq num ที่ 11



ออกสอบ



TCP ไม่ได้นับ next เป็น package num ว่าส่งไปครั้งที่ ... แต่จะนับจากใน string ส่งไปก็ byte แล้วค่อยทำการบวกเพิ่มเข้าไป

- Timeout

TCP ต้องมีการ retransmit ข้อมูล

ถ้า set สั้นไปเกิดการ retransmit เยอะ เพราะ timeout ก่อนแล้วคนรับยังไม่ได้รับ แต่มันจะคิดว่า message loss ถ้ายาวไปก็ loss ดังนั้นต้องใช้ค่าที่พอดี ดังนั้นจึงมีสูตร

TCP เป็น full duplex ทั้งสองฝั่งจะสามารถเป็นได้ทั้งคนส่งและคนรับ

ในฝั่งที่เป็นคนส่ง จะต้องมีการแปะ seq num เข้าไปใน message ผู้ส่ง และพอเริ่มส่งก็ทำการ strat timeout ถ้าเกิน timeout แล้วไม่ได้รับมันก็จะทำการ retransmit แล้วมันก็จะ start timeout เพื่อทำการส่งใหม่

ถ้าได้รับ ACK ก็ทำการ update ว่าที่ส่งไปเค้าได้รับถึงไหน แล้ว next package ก็จะได้ส่งอันถัดไป ไม่ต้องส่งซ้ำ 2 อัน

WORKING OF GO-BACK-N ARQ

Diagram illustrating the Go-Back-N ARQ protocol. The sender buffer contains frames 0 through 10. The receiver buffer contains frames 0 through 2. The sliding window is shown at the sender side, covering frames 2, 3, 4, and 5. The receiver buffer is shown with frames 0, 1, and 2. The text "Go-Back to 2" indicates the action taken when a frame is received out of order.

Window Size: 4

The diagram illustrates the Go-Back-N ARQ protocol. It shows a sender buffer with frames 0 through 10. A sliding window of size 4 is shown at the sender side, covering frames 2, 3, 4, and 5. The receiver buffer is shown with frames 0, 1, and 2. Arrows indicate the transmission of frames from the sender to the receiver. Frame 2 is marked with an 'X' and labeled 'Go-Back to 2', indicating that the receiver has received a duplicate of a frame already received.

The diagram consists of two parts, each showing a sequence of events between Host A and Host B.

lost ACK scenario:

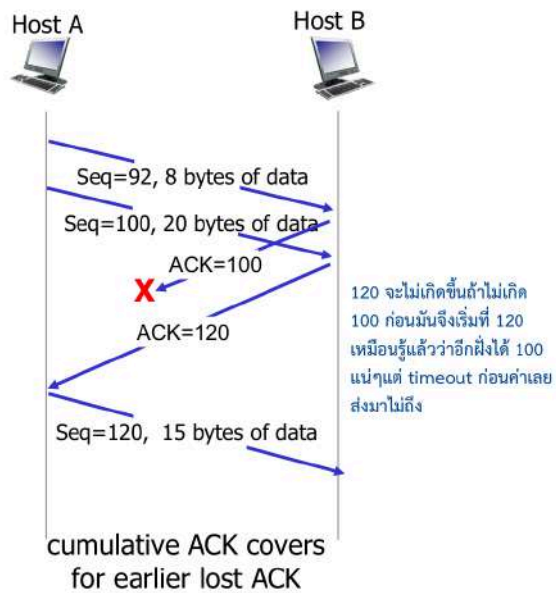
- Host A sends Seq=92, 8 bytes of data to Host B.
- Host B receives it and calculates ACK=100 = 92 + 8.
- A vertical line labeled 'timeout' occurs on Host A's side.
- Host A sends a red 'X' labeled 'timeout => resend ตัวที่ยังไม่ได้ข้อมูล'.
- Host A resends Seq=92, 8 bytes of data.
- Host B receives it and sends ACK=100.

premature timeout:

- Host A starts with SendBase=92 and sends Seq=92, 8 bytes of data.
- Host B receives it and sends ACK=100.
- Host A receives ACK=100 and updates SendBase=100.
- Host A sends Seq=100, 20 bytes of data.
- Host B receives it and sends ACK=120.
- Host A receives ACK=120 and updates SendBase=120.
- A vertical line labeled 'timeout' occurs on Host A's side.
- Host A resends Seq=92, 8 bytes of data.
- Host B receives it and sends ACK=120.
- Host A receives ACK=120 and updates SendBase=120.

Additional text on the right side of the 'premature timeout' diagram explains: 'send cumulative ACK for 120', 'resend ตัวที่ค่าได้รับแล้ว แต่เวลาส่ง ACK ด้วยความที่ได้รับถึง 120 แล้ว เลขส่ง ACK 120 กลับมา แล้ว ignore 100'.

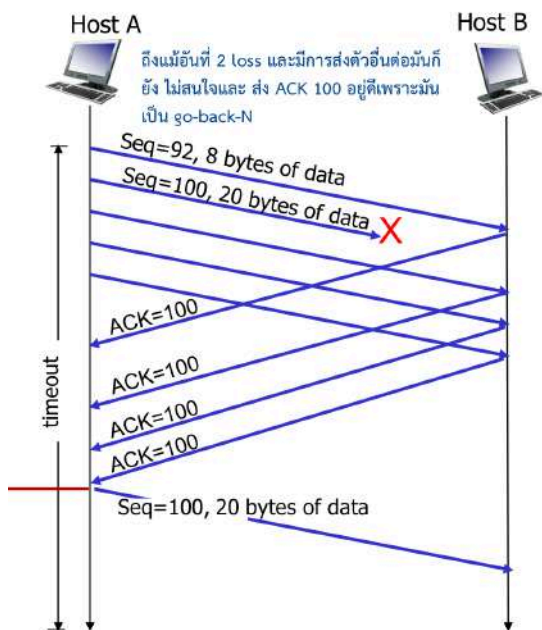
send cumulative
ACK for 120
resend ตัวที่เค้าได้รับแล้ว
แต่เวลาส่ง ACK ด้วยความ
ที่ได้รับถึง 120 แล้ว เลขส่ง
ACK 120 กลับมา แล้ว
ignore 100



- TCP fast retransmit มีเฉพาะ TCP

เป็น technique หนึ่งของ TCP ที่ไม่ต้องรอ timeout ถ้ามันได้รับค่าอะไรบางอย่างพร้อมกัน ACK ซ้ำกันเป็นครั้งที่ 3 จะ retransmit ตัวใหม่โดยไม่ต้องรอ timeout

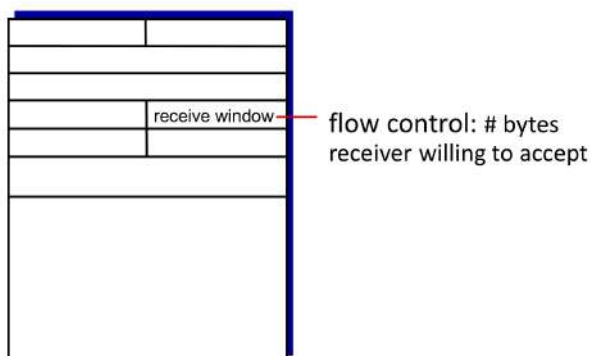
มีกฎกำหนดไว้ว่าถ้า ACK ซ้ำกันของที่มีอยู่ 3 ครั้งจะ retransmit ตัวใหม่โดยไม่ต้องรอ timeout



- TCP flow control

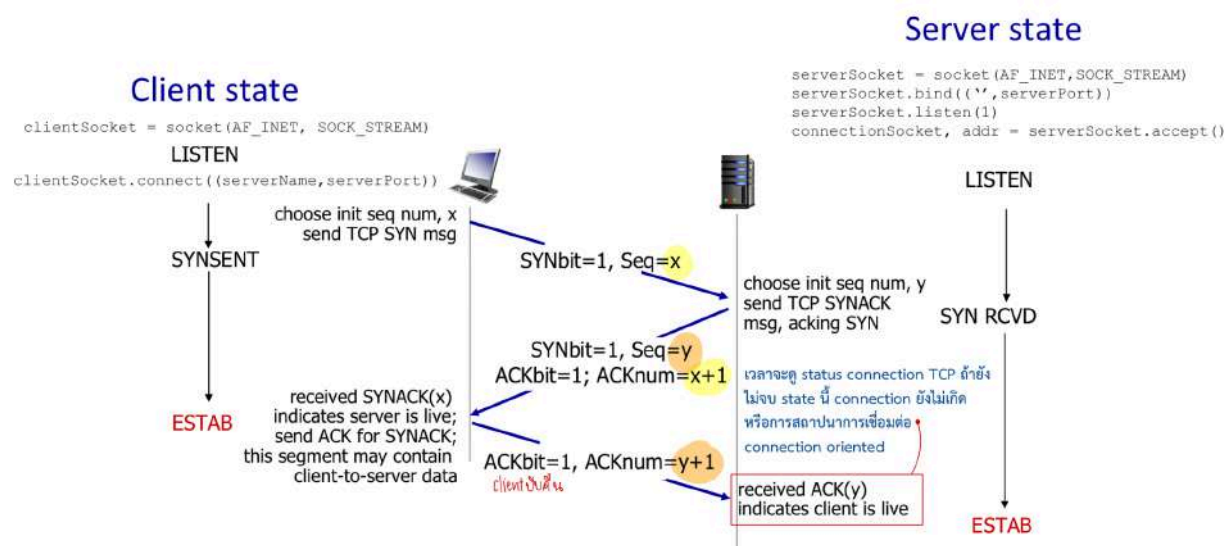
ฝั่ง receiver TCP socket ต้องมีการจอง memory buffer ไว้ ตอนรับมาต้องเก็บไว้ใน buffer ก่อนแล้วค่อยแยกไปตาม app และต้องค่อยๆ control ส่งในปริมาณที่เหมาะสมเพื่อป้องกันการ overflow

ดังนั้น เวลาที่ sender กับ receiver จะต้องส่งตาม window size หรือเป็นการ control sender ว่าไม่ให้ส่งข้อมูลมากเกินไปตามทีระบุ



- TCP connection management

ในการส่งข้อมูล TCP จะเริ่มส่ง 3 message ก่อนว่าค่า seq num , window size เป็นเท่าไร เรียกว่า connection establishment หรือ 3-way handshake -client ยื่นมือ (เรียกว่า TCP SYNbit = 1) -server ยื่นกลับ (TCP ACKbit = 1 เพื่อบอกว่าตอบกลับแล้ว) -ทั้งสองบีบมือ จากนั้นค่อยเริ่มส่งข้อมูลกัน



พอเราเปิด connection แล้วก็ต้องมีการปิด โดย client จะทำการส่ง finish bit (FINbit = 1) เพื่อบอก server ว่าต้องการปิด connection -ไม่ออกสอบ FINbit-

- Congestion VS flow control

Flow control : เป็นการ control คู่ของ connection

Congestion : มองในมุมใหญ่ที่มีหลายคนใช้ network ทุกคนต้องการส่งข้อมูล ทำให้ package มากองที่ถนน ก็ต้องทำการจัดเส้นทางให้สามารถถึงปลายทางก่อน drop package

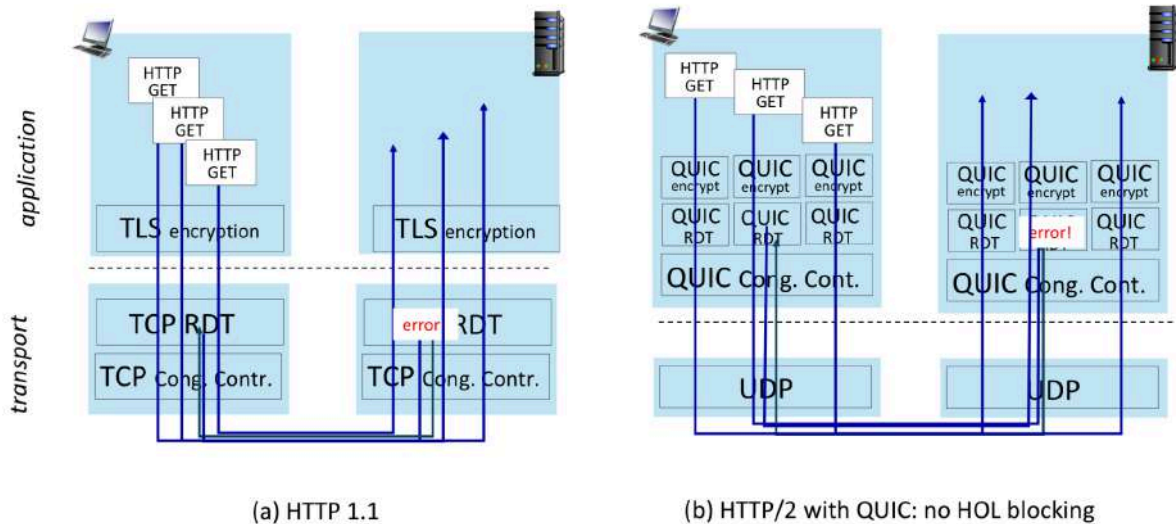
- Evolving transport-layer functionality

TCP และ UDP เป็นพื้นฐานของ transport protocol มา 40 ปี ซึ่ง TCP ก็ถูกใช้ในหลายงาน เช่น data centre network , traffic flow ดังนั้นบางทีการออกแบบอาจจะไม่ตรงกับความต้องการของ requirement สมัยใหม่ เข้าเกิน แต่ทำอะไรไม่ได้ในช่วงนั้น เลยจำเป็นต้องใช้

ถ้า Application ที่ run บน Protocol เป็น QUIC จะ provide reliable transfer

- QUIC : Quick UDP Internet Connection

เป็น Application layer protocol บน UDP เพื่อเพิ่มประสิทธิภาพของ HTTP ให้เป็น HTTPSs ที่เร็วขึ้น
เนื่องจาก TCP & TLS มีการทำ handshake ดังนั้น QUIC เลยรวบให้การ handshake เหลือแค่อันเดียว

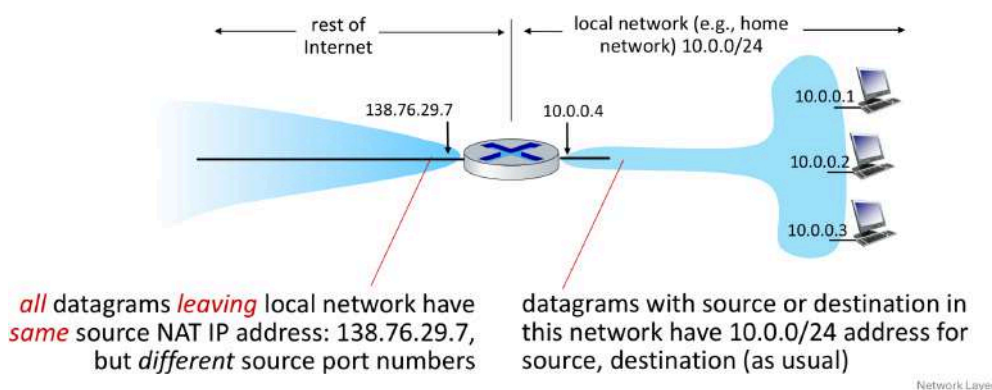


จะดูยังไงว่าใช้ TLS หรือ QUIC บน browser ?

- Something more about layer 3

layer 3 มีหน้าที่สำคัญอีก นอกจากการกำหนดหมายเลข ip, subnet แต่ layer 3 router จะเป็นคนเลือกเส้นทางว่าจะเดินไปเส้นทางไหน ซึ่งเป็นหน้าที่ที่สำคัญมาก ซึ่ง router จะทำงานให้ แต่เราก็จะต้องทำการ config router นี้เช่นกัน

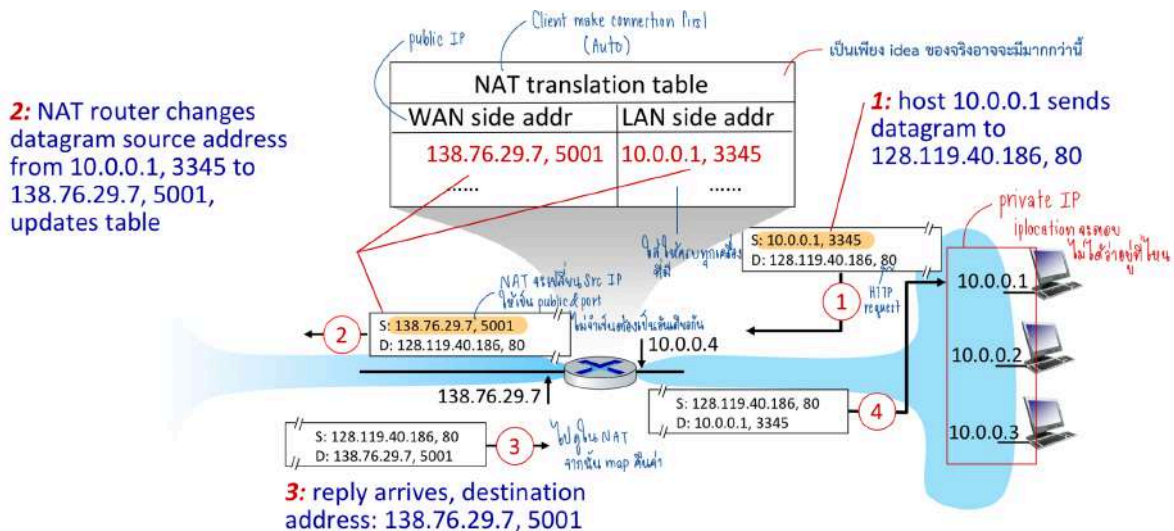
- เหตุผล ที่ต้องมี public ip address คือ private ip address ไม่พอ ซึ่งเวลาที่ส่งข้อมูลกลับมาหา client router จะรู้ว่าต้องส่งให้ใครผ่าน NAT
- NAT : network address translation



เวลาคืนค่ากลับมาหา Client router สามารถรู้ได้จากค่า NAT

NAT มีหน้าที่ในการ map public ip กับ private ip เข้าด้วยกันเนื่องจาก private ip ไม่พอ ซึ่งคนข้างนอกจะรู้จักเราผ่าน WAN interface public IP ตัว router ไม่ได้มีหน้าที่แค่เลือกเส้นทางอย่างเดียว แต่ต้องทำการ mapping ระหว่างข้างในกับข้างนอก

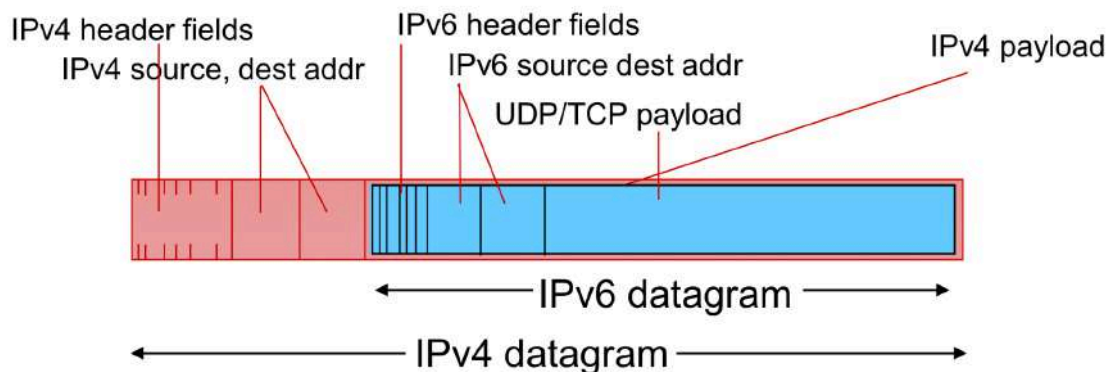
public ip address เวลา search หาจบออก geolocation และทุกคนที่อยู่ข้างนอกจะเห็นว่า 3 เครื่องข้างบนมาจาก IP เดียวกัน ดังนั้น NAT ต้องทำหน้าที่ในการคืนให้ถูกกว่าถ้าคืนกลับแล้วต้องคืนให้เครื่องไหน ข้อดีคือเรามี public IP แคหมายเลขเดียวพอ จากนั้นค่อยไป manage network ข้างใน



แต่ในองค์กรใหญ่ๆอาจจะมี NAT หลายตัวจากหลายๆ IP เช่น internet ที่มหาศาล มันเลยดูเหมือนเป็นคนละเครื่อง

การมี NAT เลยช่วยให้คนไม่กังวลเรื่องหมายเลข IP ไม่พอ ซึ่งถ้าใช้ IPv6 จะไม่เกิดเหตุการณ์ IP ไม่พอใช้ แต่ด้วย technology ของ NAT มันดีมาก ทำให้องค์กรไม่อยากจะเปลี่ยน เลยเปลี่ยนเป็น IPv6 ไม่ได้

IPv6 ช่วยแก้ไขปัญหของ IPv4 หลายๆอย่างเช่น เพิ่ม security, แต่ละเครื่องมีเลข IP ของตัวเอง แต่ก็มีส่วนที่ใช้ IPv4 บางส่วนใช้ IPv6 ซึ่งก็สามารถติดต่อกันได้ด้วยการยัด IPv6 ไปใส่ใน source dest ของ IPv4 หรือที่เรียกว่า tunneling



บริษัท ABC ไปจดทะเบียน IP address แล้วได้ network address เป็น 161.246.0.0/20

ทางบริษัทนี้มีการวางแผน network โดยดังนี้

1. คาดหวังว่าจะมีการขยายแผนกมากที่สุด 7 แผนก
2. กำหนดให้มี guest network สำหรับ คนนอกที่เข้ามาเยี่ยมชม 1 network
3. สร้าง network wifi (ไม่เกี่ยวข้องกับแผนก) 2 network

$$7 + 1 + 2 = 10 \text{ nw}$$

โจทย์ข้อ A

1. คนออกแบบ

1.1 ต้องออกแบบระบบให้มีทั้งหมด ที่ subnet 10 subnet

1.2 แต่ละ subnet network address คืออะไร

1.3 แต่ละ subnet broadcast address คืออะไร

1.4 แต่ละ subnet เบอร์ IP แรก และ IP สุดท้ายคืออะไร

1.5 แต่ละ subnet สามารถต่อเครื่องคอมพิวเตอร์ ได้กี่เครื่อง $2^5 - 2 = 254$

แบ่งออกเป็น 10 subnet

192.168.1.0 /24

255.255.255.0

11111111.11111111.11111111.00000000

Subnet into 5 networks

① Determine network bits and bit

⑤ → 1 → 1 → 1 = 3 bits

128	64	32	16	8	4	2	1

∴ 3 bits

② Hack the host bits

11111111.11111111.11111111.11100000

= 255.255.255.224

192.168.1.0/27

③ Determine the subnet

$2^5 = 32$

④ Write it out

192.168.1.0 - 192.168.1.31

192.168.1.32 - 192.168.1.63

192.168.1.64 - 192.168.1.95

192.168.1.96 - 192.168.1.127

192.168.1.128 - 192.168.1.159

Given a host IP address of 48.25.24.71/21, what is the broadcast address for the subnetwork? (Select the best answer.)

☐ A. 48.25.25.254

☐ B. 48.25.24.240

☐ C. 48.25.31.254

☐ D. 48.25.31.255

☐ E. 48.25.24.255

☐ F. 48.25.24.127

IP = 48.25.24.71

mask = 11111111.11111111.11110000.00000000
= 255.255.240.0

IP = 00110000.00011001.00011000.01000111

IP & Mask

00110000.00011001.00011000.01000111
11111111.11111111.11111000.00000000
00110000.00011001.00011000.01000111

Broadcast Address = 00110000.00011001.00011111.11111111
= 48.25.31.255

broadcast Address = 00110000.00011001.00011111.11111111
= 48.25.31.255

บริษัท ABC ไปจดทะเบียน IP address แล้วได้ network address เป็น

161.246.0.0 / 20

ทางบริษัทนี้มีการวางแผนเน็ตเวิร์ค โดยค ดังนี้

- (1) คาดหวังว่าจะมีการขยาย แผนก มากที่สุด 7 แผนก
- (2) กำหนดให้มี guest network สำหรับ คนนอกที่เข้ามาเยี่ยมชม 1 เน็ตเวิร์ค
- (3) สร้าง network wifi (ไม่เกี่ยวข้องกับแผนก) 2 เน็ตเวิร์ค

โจทย์ข้อ A

1. คนออกแบบ

- 1.1 ต้องออกแบบระบบให้มีทั้งหมด ที่ subnet 10 subnet
- 1.2 แต่ละ subnet network address คือ อะไร
- 1.3 แต่ละ subnet broadcast address คืออะไร
- 1.4 แต่ละ subnet เบอร์ IP แรก และ IP สุดท้ายคืออะไร
- 1.5 แต่ละ subnet สามารถต่อเครื่องคอมพิวเตอร์ ได้กี่เครื่อง $2^5 - 2 = 254$ เครื่อง

Subnet Mask : 1111111.1111111.11110000.00000000

① นับว่าต้องเพิ่ม network bit อีก bit

ต้องการ 10 subnet

128 64 32 16 8 4 2 1
10 → → → → = 4 bit
∴ 4 bit

② ทำการหา subnet

1111111.1111111.1111111.00000000

255.255.255.0 หรือ /24

∴ 161.246.0.0 / 24

③ นับว่าจำนวน IP ในแต่ละ subnet มีกี่ IP

1111111.1111111.1111111.00000000

จำนวน IP = $2^8 = 256$ (รวม network & broadcast)

④ เขียน

subnet 0:	161.246.0.0	161.246.0.255
subnet 1:	161.246.1.0	161.246.1.255
subnet 2:	161.246.2.0	161.246.2.255
subnet 3:	161.246.3.0	161.246.3.255
subnet 4:	161.246.4.0	161.246.4.255
subnet 5:	161.246.5.0	161.246.5.255
subnet 6:	161.246.6.0	161.246.6.255
subnet 7:	161.246.7.0	161.246.7.255
subnet 8:	161.246.8.0	161.246.8.255
subnet 9:	161.246.9.0	161.246.9.255

↓ ↓
network address broadcast address